Arquitectura NET: La plataforma.Net. Modelo de programación. Servicios. Herramientas. Persistencia. Seguridad

ÍNDICE

1 LAS APLICACIONES DISTRIBUIDAS 4

1.1 ARQUITECTURA 4
1.2 PRINCIPIOS DE DISEÑO 5
1.2.1 REQUERIMIENTOS FUNCIONALES 5
1.2.2 REQUERIMIENTOS OPERATIVOS 6
1.3 SERVICIOS WEB 7

2 LA PLATAFORMA .NET 8

2.1 .NET FRAMEWORK 10
2.1.1 COMMON LANGUAGE RUNTIME (CLR) 11
2.1.2 ENSAMBLADOS 14
2.1.3 BIBLIOTECA DE CLASES ESTÁNDAR 15
2.1.4 ACCESO A DATOS 16
2.1.5 ASP.NET 20
2.1.5.1 Windows forms frente a Web forms 22
2.1.6 WINDOWS PRESENTATION FOUNDATION 23
2.1.7 WINDOWS COMMUNICATION FOUNDATION 24
2.1.8 LINQ 25
2.1.9 HERRRAMIENTA DESARROLLO: MS VISUAL STUDIO.NET 28
2.1.10 VERSIONES DE LA PLATAFORMA .NET 29

3 MODELO DE PROGRAMACIÓN 32

3.1 INTRODUCCIÓN 32
3.1.1 ASP.NET WEB FORMS 32
3.1.2 ASP.NET MVC 33
3.1.3 ASP.NET WEB PAGES 34
3.2 LENGUAJES DE PROGRAMACIÓN 34

4 SERVICIOS DE AZURE 35

4.1 INTRODUCCIÓN 35
4.2 HOSPEDAJE DE APLICACIONES 36
4.2.1 MÁQUINAS VIRTUALES 36
4.2.2 CONTENEDORES 36
4.2.3 HOSPEDAJE DE CONTENEDORES CON AZURE CONTAINER INSTANCES 36
4.2.4 HOSPEDAJE DE CONTENEDORES CON AZURE CONTAINER SERVICE 36
4.2.5 AZURE FUNCTIONS 36
4.2.6 AZURE LOGIC APPS 37
4.2.7 AZURE APP SERVICE 38
4.3 AZURE CONTENT DELIVERY NETWORK 38
4.4 AZURE REDIS CACHE 38
4.5 AZURE TRAFFIC MANAGER 39

4.6 AZURE SQL DATABASE 39
4.7 AZURE DATABASE FOR MYSQL 40
4.8 AZURE DATABASE FOR POSTGRESQL 40
4.9 AZURE COSMOS DB 41
4.10 AZURE STORAGE 41
4.10.1 BLOB STORAGE 41
4.10.2 QUEUE STORAGE 42
4.10.3 FILE STORAGE 42
4.10.4 DISK STORAGE 42
4.11 AZURE DATA LAKE STORE 42
4.12 AZURE SQL DATA WAREHOUSE 43
4.13 BÚSQUEDA DE AZURE 43
4.14 AZURE COGNITIVE SERVICES 44
4.15 AZURE SERVICE BUS 44
4.16 AZURE EVENT HUBS 45
4.17 AZURE IOT HUB 45
4.18 AZURE EVENT GRID 45
4.19 AZURE ACTIVE DIRECTORY 46
4.20 AZURE KEY VAULT 46

5 HERRAMIENTAS 47

6 PERSISTENCIA 49

7 SEGURIDAD 53

7.1 INTRODUCCIÓN 53
7.2 SEGURIDAD ASP.NET DE APLICACIONES WEB 54

8 DESARROLLO DE APLICACIONES WEB EN .NET 55

8.1 ARQUITECTURA DETALLADA DE UNA APLICACIÓN WEB DISTRIBUIDA 55

1 Las aplicaciones distribuidas

1.1 Arquitectura

En el desarrollo de las aplicaciones se ha ido produciendo una evolución en el diseño. Inicialmente, las aplicaciones se diseñaban como un módulo único, donde se integraban todos los elementos conjuntamente.

Posteriormente, se pasó a un modelo cliente servidor, en dos capas, con la presentación en el lado del cliente (y una cierta funcionalidad asociada), y la lógica de negocio y el acceso a los datos en el lado del servidor.

La tendencia actual separa las aplicaciones en tres o más capas, evolucionando también en el concepto de acoplamiento, al pasar de un diseño de módulos muy acoplados, a un diseño de módulos cada vez más desacoplados.

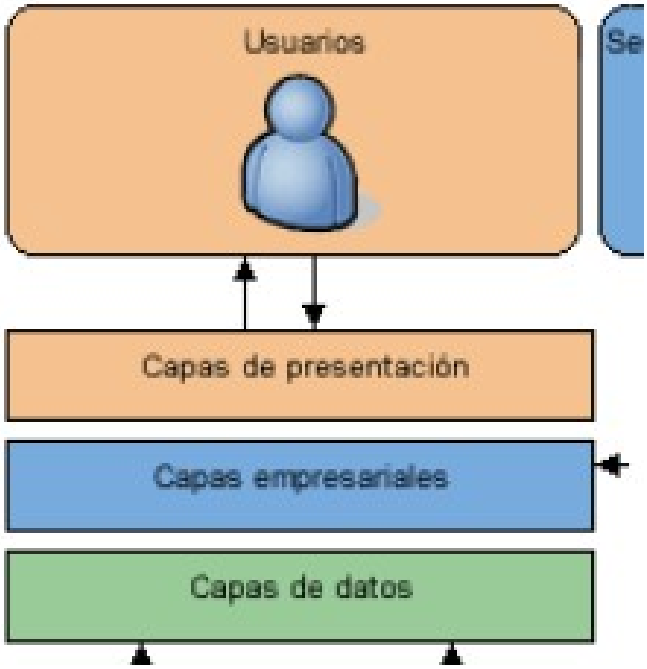
La separación típica en tres capas presenta las siguientes características:

Capa de presentación: Encargada de mantener la interactividad con el usuario. Puede desarrollarse con un cliente rico y pesado (como Windows forms) o con un cliente ligero, típicamente un navegador de Internet. La capa de presentación contiene la interfaz de usuario propiamente dicha, y puede tener también procesos de interfaz de usuario, dependiendo del tipo de aplicación. Suelen basarse en una capa de 'Componentes de Control de usuario'.

Capa de lógica de negocio: La capa de negocio incluye toda la lógica de la aplicación, realizada a través de componentes y entidades empresariales, workflows e interfaces de servicios. Esta capa está soportada por Servidores de Aplicaciones (con aplicaciones) y Servicios (de BBDD, de Directorio, de Seguridad, etc.).

Capa de acceso a datos: En esta capa se gestiona todo el acceso a datos, tanto internos (de la propia aplicación) como externos, implementados a través de agentes de servicios. Para ello, se puede usar ADO.NET, OleDB y XML entre otros.

A continuación, se muestra una visión simplificada de una aplicación y sus capas:



Afectando a todo el conjunto de la aplicación (a todas sus capas), se encuentran las políticas de seguridad, de administración y de comunicaciones.

Dentro de la directiva de seguridad, se deben implementar las cuestiones relacionadas con la autenticación de los usuarios, la autorización para el acceso a los recursos, el establecimiento de perfiles de usuario, y las auditorías de los sistemas, para realizar el seguimiento de los usuarios y de la actividad empresarial.

Referente a la administración, se debe definir la configuración óptima del sistema, la monitorización del mismo, para permitir detectar posibles cuellos de botella, la gestión de excepciones que ocurren en la aplicación, y la publicación de servicios.

Por último, en la directiva de comunicaciones se definirá el modelo de comunicación de la aplicación (síncrono vs. paso de mensajes), los protocolos empleados y se tendrá en cuenta el ancho de banda que puede requerir la aplicación.

1.2 Principios de diseño

Entre los principios de diseño de una aplicación, se presentan una serie de requerimientos funcionales y una serie de requerimientos operativos.

1.2.1 Requerimientos funcionales

La aplicación se debe diseñar para cumplir los requisitos funcionales. Entre ellos se encuentran los siguientes:

Funcionalidad: La aplicación debe cumplir la funcionalidad establecida en el diseño.

Accesibilidad: La aplicación debe diseñarse para cumplir los requisitos de accesibilidad impuestos en las normas WAI.

Flexibilidad: La aplicación debe diseñarse de manera que sea abierta y flexible, y permita la introducción de nueva funcionalidad de manera sencilla.

1.2.2 Requerimientos operativos

En el desarrollo de la aplicación se deben emplear las técnicas de diseño y las prácticas que permitirán obtener los requerimientos operativos (no funcionales) para la aplicación y los servicios. Los requerimientos operativos más importantes son los siguientes:

Escalabilidad: La escalabilidad de una aplicación es la capacidad de la misma para proporcionar un nivel aceptable de rendimiento general cuando aumentan uno o varios factores de carga. Entre estos factores se incluyen el número de usuarios, la cantidad de datos que administra la aplicación, así como el número de transacciones.

Disponibilidad: La disponibilidad es una medida del porcentaje de tiempo en el que la aplicación puede responder a las solicitudes de modo que los autores de la llamada estén en espera. Ocasionalmente incluso las aplicaciones más sólidas no están disponibles, aunque la aplicación se debe diseñar de forma que el riesgo de las interrupciones inesperadas se vean reducidas. Para las aplicaciones empresariales importantes, muchas organizaciones tienen como objetivo alcanzar los "cinco nueves" o 99,999% de disponibilidad; este nivel de solidez requiere un detallado diseño y planeamiento.

Capacidad de mantenimiento: En relación con el mantenimiento, la aplicación se debe diseñar e implementar de modo que se pueda mantener y reparar con facilidad. Se debe tener en cuenta la estructuración del código de forma previsible, con el empleo de componentes o de técnicas de codificación.

Seguridad: La seguridad siempre ha sido un aspecto fundamental en el diseño de una aplicación, especialmente cuando la misma se expone en el Web. En gran parte, las decisiones que se adopten en relación a la seguridad dependerán de la directiva de seguridad.

Facilidad de uso: Las aplicaciones se deben diseñar con una interfaz intuitiva, que permita aprender el uso de la aplicación sin necesidad de utilizar un complejo manual.

Rendimiento: El rendimiento del servicio y la aplicación es un factor clave en una buena experiencia del usuario y en la utilización eficaz del hardware. Mientras que el rendimiento es un atributo que se puede mejorar ajustando la implementación y el código del sistema una vez creado, es importante considerarlo en las fases de diseño y arquitectura.

Fiabilidad: La aplicación debe comportarse de forma predecible y fiable. Para lograr este requisito, es conveniente el empleo de componentes ya desarrollados y empleados en otras aplicaciones (o el empleo de librerías estándar).

Administrable: El gran número de potenciales usuarios y sus características de disponibilidad, implican que la aplicación se debe diseñar de manera que sea fácilmente administrable.

1.3 Servicios web

Un servicio web es un servicio, con un interfaz definido y conocido, al que se puede acceder a través de internet.

Los web services son componentes software que permiten a los usuarios usar aplicaciones de negocio que comparten datos con otros programas modulares, vía Internet. Son aplicaciones independientes de la plataforma que pueden ser fácilmente publicadas, localizadas e invocadas mediante protocolos web estándar, como XML, SOAP, UDDI o WSDL. El objetivo final es la creación de un directorio online de web services, que pueda ser localizado de un modo sencillo y que tenga una alta fiabilidad.

La funcionalidad de los protocolos empleados es la siguiente:

- ✓ XML (eXtensible Markup Language): Un servicio web es una aplicación web creada en XML.
- ✓ WSDL (Web Services Description Language): Este protocolo se encarga de describir el web service cuando es publicado. Es el lenguaje XML que los proveedores emplean para describir sus web services. Un documento WSDL proporciona la información necesaria al cliente para interaccionar con el servicio Web.
- ✓ SOAP (Simple Object Access Protocol): Permite que programas que corren en diferentes sistemas operativos se comuniquen. La comunicación entre las diferentes entidades se realiza mediante mensajes que son rutados en un sobre SOAP. SOAP proporciona un mecanismo estándar de empaquetar un mensaje. Un mensaje SOAP se compone de un sobre que contiene el cuerpo del mensaje y cualquier información de cabecera que se utiliza para describir el mensaje.
- ✓ UDDI (Universal Description Discovery and Integration): Este protocolo permite la publicación y localización de los servicios. Los directorios UDDI actúan como una guía telefónica de web services.

Aunque la idea de la programación modular no es nueva, el éxito de esta tecnología reside en que se basa en estándares conocidos en los que ya se tiene una gran confianza, como el XML.

En el desarrollo de servicios web se pueden distinguir los siguientes elementos:

- ➢ Servicio Web: Aplicación web a utilizar. Descrita mediante WSDL. Registrada en el registro UDDI para poder ser usada
- ➢ Proveedor del Servicio: Plataforma que aloja el servicio físicamente (Ej: IIS)
- ➢ Registro de servicios: contiene la descripción y los detalles de uso de los servicios

- Solicitante del servicio: La aplicación o usuario que usa el servicio, mediante llamadas SOAP.

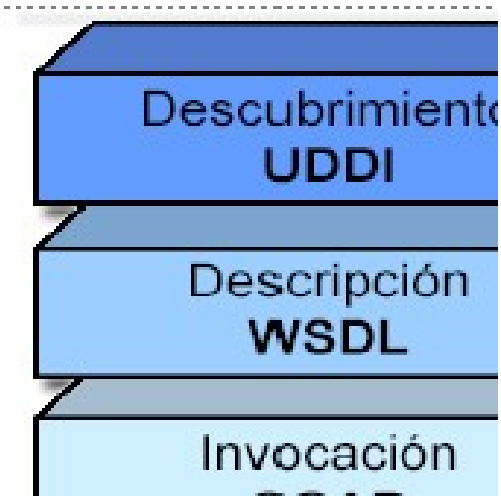
Las características de los servicios web son, por tanto, las siguientes:

- ✓ Cada Servicio Web cubre una funcionalidad o grupo concreto de funcionalidades
- ✓ Los servicios exponen las reglas de negocio (publican la funcionalidad)
- ✓ Usan un protocolo transparente de acceso (lo más sencillo posible, HTTP)
- ✓ Interoperabilidad: Permitir la interacción de aplicaciones distribuidas

El éxito de los servicios web descansa en las siguientes premisas:

- Basado en estándares abiertos: independencia tecnológica (plataforma, lenguaje, etc.)
- Usa tecnologías probadas (HTTP, XML, etc) -> fácil adopción
- Facilidad de uso
- Facilita en gran medida la reutilización
- Muy adecuado para el B2B

En el siguiente esquema se puede ver el conjunto de protocolos empleados:



2 La plataforma .NET

La plataforma .NET es un conjunto de tecnologías diseñadas para transformar Internet en una plataforma informática distribuida a escala completa. Proporciona nuevas formas de desarrollar aplicaciones a partir de colecciones de Servicios Web. La plataforma .NET soporta totalmente la infraestructura existente de Internet, incluyendo HTTP, XML y SOAP.



La plataforma está constituida por un conjunto de componentes, incluyendo el .NET Framework, los servicios .NET Building Blocks, los servidores .NET Enterprise Servers y Microsoft Visual Studio® .NET. El objetivo de la plataforma .NET es simplificar el desarrollo Web proporcionando todas las herramientas y tecnologías necesarias para desarrollar aplicaciones Web distribuidas.

La plataforma .NET proporciona:

- ✓ Un modelo de programación coherente e independiente del lenguaje para todas las capas o niveles de una aplicación.
- ✓ Una interoperabilidad transparente entre tecnologías.
- ✓ Una fácil migración desde tecnologías existentes.
- ✓ Un completo soporte de tecnologías de Internet independientes de la plataforma y basadas en estándares, incluyendo Hypertext Transfer Protocol (HTTP), Extensible Markup Language (XML) y Simple Object Access Protocol (SOAP).

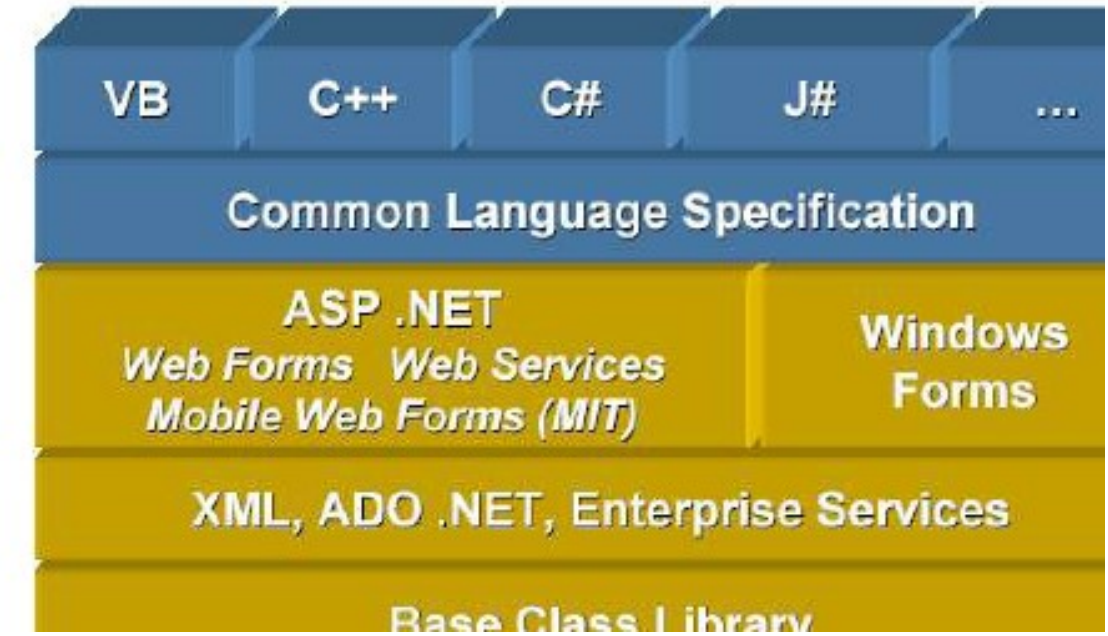
El componente central de la plataforma es el .NET Framework.

2.1 .NET Framework

Antes del modelo COM, las aplicaciones estaban formadas por entidades totalmente separadas con poca o ninguna integración. Al utilizar COM, es posible integrar componentes dentro de una aplicación y entre aplicaciones exponiendo una serie de interfaces comunes. Sin embargo, todavía es necesario que los desarrolladores escriban el código para envolver, gestionar e integrar componentes y objetos.

.NET Framework es un componente integral de Windows que admite la creación y la ejecución de la siguiente generación de aplicaciones y servicios Web XML.

Desde un punto de vista esquemático, el .net framework podría considerarse constituido por los siguientes elementos:



En el .NET Framework, los componentes se construyen sobre una base común. Ya no es necesario escribir código para permitir que los objetos interactúen directamente entre sí. En el entorno .NET, ya no es necesario escribir envoltorios de componentes, puesto que los componentes no los utilizan. El .NET Framework puede interpretar las construcciones a los que los desarrolladores están acostumbrados utilizando lenguajes orientados a objetos. El .NET Framework soporta totalmente clases, herencia, métodos, propiedades, eventos, polimorfismo, constructores y otras construcciones orientadas a objetos.

Como características comunes pueden citarse las siguientes:

- ✓ Proporciona todas las herramientas y tecnologías necesarias para construir aplicaciones web distribuidas.
- ✓ Presenta un modelo de programación consistente para ser utilizado en todas las capas de una aplicación.
- ✓ Proporcionar un entorno coherente de programación orientada a objetos, en el que el código de los objetos se pueda almacenar y ejecutar de forma local, ejecutar de forma local pero distribuida en Internet o ejecutar de forma remota.
- ✓ Proporcionar un entorno de ejecución de código que reduzca lo máximo posible la implementación de software y los conflictos de versiones.
- ✓ Ofrecer un entorno de ejecución de código que fomente la ejecución segura del mismo, incluso del creado por terceras personas desconocidas o que no son de plena confianza.
- ✓ Proporcionar un entorno de ejecución de código que elimine los problemas de rendimiento de los entornos en los que se utilizan secuencias de comandos o intérpretes de comandos.
- ✓ Ofrecer al programador una experiencia coherente entre tipos de aplicaciones muy diferentes, como las basadas en Windows o en el Web.
- ✓ Basar toda la comunicación en estándares del sector para asegurar que el código de .NET Framework se puede integrar con otros tipos de código, haciendo un uso intensivo de http o XML.

Dentro del framework destacan los siguientes elementos:

2.1.1 Common Language Runtime (CLR)

El CLR es el entorno donde se ejecutan todas las aplicaciones .NET. El CLR es el verdadero núcleo del Framework de .Net, ya que es el entorno de ejecución en el que se cargan las aplicaciones desarrolladas en los distintos lenguajes, ampliando el conjunto de servicios que ofrece el sistema operativo estándar.

A la hora de desarrollar aplicaciones, éstas pueden estar constituidas por código administrado o código no administrado.

- ✓ Código administrado es un término que describe al código de una aplicación que corre o se ejecuta bajo un ambiente administrado. Esto significa que el código de la aplicación no corre directamente sobre el sistema operativo del dispositivo, sino que en vez de ello, se apoya de un ambiente de ejecución (runtime engine) para ser ejecutado (en este caso, el CLR). El ambiente de ejecución se encarga de asignar recursos y servicios de soporte como seguridad, administración de la memoria, etc., por lo que decimos que la aplicación está siendo administrada por el ambiente de ejecución. Contar con un ambiente administrado permite tener aplicaciones que son independientes tanto del procesador como del sistema operativo.

- ✓ Código no administrado es aquel que al compilarse genera un binario que es ejecutado directamente por la máquina, fuera de algún ambiente de ejecución, y que por tanto no obtiene servicios de soporte a través del mismo. En general, los proyectos estarán desarrollados con código administrado.

Cuando se compila a código administrado, el compilador convierte el código fuente en un Lenguaje intermedio de Microsoft (MSIL), que es un conjunto de instrucciones independiente de la CPU que se pueden convertir de forma eficaz en código nativo. MSIL incluye instrucciones para cargar, almacenar, inicializar y llamar a métodos en los objetos, así como instrucciones para operaciones lógicas y aritméticas, flujo de control, acceso directo a la memoria, control de excepciones y otras operaciones. Antes de poder ejecutar código, se debe convertir MSIL al código específico de la CPU, normalmente mediante un compilador Just-In-Time (JIT). El CLR proporciona uno o varios compiladores JIT para cada arquitectura de equipo compatible, por lo que se puede compilar y ejecutar el mismo conjunto de MSIL en cualquier arquitectura compatible.

Cuando el compilador produce MSIL, también genera metadatos. Los metadatos describen los tipos que aparecen en el código, incluidas las definiciones de los tipos, los miembros a los que se hace referencia en el código y otros datos que el motor de tiempo de ejecución utiliza en tiempo de ejecución.

Como parte de la compilación MSIL en código nativo, el código debe pasar un proceso de comprobación, a menos que el administrador haya establecido una directiva de seguridad que permita al código omitir esta comprobación. En esta comprobación se examina el MSIL y los metadatos para determinar si el código garantiza la seguridad de tipos, lo que significa que el código sólo tiene acceso a aquellas ubicaciones de la memoria para las que está autorizado. La seguridad de tipos ayuda a aislar los objetos entre sí y, por tanto, ayuda a protegerlos contra daños involuntarios o maliciosos.

El motor de tiempo de ejecución se basa en el hecho de que se cumplen ciertas condiciones de seguridad, como por ejemplo que la referencia a un tipo es estrictamente compatible con el tipo al que hace referencia, o que en un objeto sólo se invocan las operaciones definidas adecuadamente.

A la hora de ejecutar el código, el CLR proporciona la infraestructura que permite que la ejecución administrada tenga lugar, así como una gran cantidad de servicios que se pueden utilizar durante la ejecución. Durante la ejecución, el código administrado recibe servicios como la recolección de elementos no utilizados, seguridad, interoperabilidad con código no administrado, compatibilidad de depuración entre lenguajes diferentes y compatibilidad mejorada con el control de versiones y la implementación.

La administración de memoria automática es uno de los servicios que proporciona el CLR durante la ejecución administrada. El recolector de elementos no utilizados del CLR administra la asignación y liberación de la memoria de una aplicación. Esto significa que los programadores no tienen que escribir código para realizar tareas de administración de

memoria al programar aplicaciones administradas. La administración automática de la memoria puede eliminar problemas frecuentes, como olvidar liberar un objeto y causar una pérdida de memoria, o intentar tener acceso a la memoria de un objeto que ya se ha liberado.

En cuanto a la asignación de memoria, cuando se inicializa un nuevo proceso, el motor de tiempo de ejecución reserva una región contigua de espacio de direcciones para el proceso. Este espacio de direcciones reservado se denomina montón administrado. El montón administrado mantiene un puntero a la dirección a la que se asignará el siguiente objeto del montón. Inicialmente, este puntero se establece en la dirección base del montón administrado. Todos los tipos de referencia se asignan en el montón administrado. Cuando una aplicación crea el primer tipo de referencia, se le asigna memoria en la dirección base del montón administrado. Cuando la aplicación crea el siguiente objeto, el recolector de elementos no utilizados le asigna memoria en el espacio de direcciones que sigue inmediatamente al primer objeto. Siempre que haya espacio de direcciones disponible, el recolector de elementos no utilizados continúa asignando espacio a los objetos nuevos de este modo.

La asignación de memoria desde el montón administrado es más rápida que la asignación de memoria no administrada. Como el tiempo de ejecución asigna memoria a los objetos agregando un valor a un puntero, este método es casi tan rápido como la asignación de memoria desde la pila. Además, puesto que los nuevos objetos que se asignan consecutivamente se almacenan uno junto a otro en el montón administrado, la aplicación puede tener un acceso muy rápido a los objetos.

Para la liberación de memoria, el motor de optimización del recolector de elementos no utilizados determina cuál es el mejor momento para realizar una recolección basándose en las asignaciones realizadas. Cuando el recolector lleva a cabo una recolección, libera la memoria de los objetos que ya no usa la aplicación. Determina qué objetos ya no se usan examinando las raíces de la aplicación.

Además, el CLR implementa los siguientes servicios:

- ✓ Cargador de Clases: Permite cargar en memoria las clases.
- ✓ Administrador de Código: Coordina toda la operación de los distintos subsistemas del Common Language Runtime.
- ✓ Motor de Seguridad: Administra la seguridad del código que se ejecuta.
- ✓ Motor de Depuración: Permite hacer un seguimiento de la ejecución del código aún cuando se utilicen lenguajes distintos.
- ✓ Verificador de Tipos: Controla que las variables de la aplicación usen el área de memoria que tienen asignado.
- ✓ Administrador de Excepciones: Maneja los errores que se producen durante la ejecución del código.

- ✓ Soporte de multiproceso (threads): Permite ejecutar código en forma paralela.

2.1.2 Ensamblados

Los ensamblados son las unidades de creación de las aplicaciones .NET Framework; constituyen la unidad fundamental de implementación, control de versiones, reutilización, ámbitos de activación y permisos de seguridad. Un ensamblado es una colección de tipos y recursos creados para funcionar en conjunto y formar una unidad lógica de funcionalidad. Los ensamblados proporcionan a Common Language Runtime la información necesaria para conocer las implementaciones de tipos.

Los ensamblados son una parte fundamental de la programación con .NET Framework. Un ensamblado realiza las funciones siguientes:

Contiene el código que ejecuta Common Language Runtime. El código del lenguaje intermedio de Microsoft (MSIL) de un archivo ejecutable portable (PE) no se ejecuta si no tiene asociado un manifiesto de ensamblado.

Crea un límite de seguridad. Un ensamblado es la unidad en la que se solicitan y conceden los permisos. Cuando se construye un ensamblado, se puede especificar el conjunto de permisos que son necesarios para la ejecución del mismo.

Crea un límite de tipos. La identidad de cada tipo incluye el nombre del ensamblado en que reside.

Crea un límite de ámbito de referencia. El manifiesto del ensamblado contiene los metadatos del ensamblado que se utilizan para resolver tipos y satisfacer las solicitudes de recursos. Especifica los tipos y recursos que se exponen fuera del ensamblado. El manifiesto también enumera otros ensamblados de los que depende.

Forma un límite de versión. El ensamblado es la unidad versional más pequeña de Common Language Runtime; todos los tipos y recursos del mismo ensamblado pertenecen a la misma versión.

Crea una unidad de implementación. Cuando se inicia una aplicación, sólo deben estar presentes los ensamblados a los que llama la aplicación inicialmente.

Los ensamblados pueden ser privados o públicos. En el caso de ser privados, se utilizan únicamente por una aplicación y se almacenan en el mismo directorio que la aplicación. En cambio, los públicos se instalan en un directorio común de ensamblados llamado global assembly cache o GAC, y son accesibles para cualquier aplicación .NET instalada en la máquina.

En general, un ensamblado está formado por cuatro elementos:

- ✓ El manifiesto del ensamblado, que contiene los metadatos del ensamblado.
- ✓ Los metadatos de tipos.
- ✓ El código de lenguaje intermedio de Microsoft (MSIL) que implementa los tipos.

- ✓ Un conjunto de recursos.

El manifiesto de ensamblado es el único elemento obligatorio, pero se necesitan o bien los tipos o bien los recursos para proporcionar al ensamblado una funcionalidad significativa.

Estos elementos se pueden agrupar en un ensamblado de varias formas. Se puede agrupar todos los elementos en un solo archivo físico, o en varios.

El manifiesto contiene el nombre del ensamblado, el número de versión, la referencia cultural y la información sobre el nombre seguro (estos elementos constituyen la identidad del ensamblado). Además, incluye la lista de todos los archivos del ensamblado (junto con su huella hash) e información sobre ensamblados a los que se hace referencia.

2.1.3 Biblioteca de clases estándar

El framework pone a disposición de los desarrolladores un conjunto de herramientas que les permite obtener una solución rápida para la mayoría de los problemas encontrados durante la realización de una aplicación.

La biblioteca de clases base son las clases sobre las cuales se construyen todas las demás clases que utilizan los programas de .NET. La clase madre de todas es System. A partir de ella por un mecanismo llamado herencia de clases, se construyen las demás clases. La biblioteca es jerárquica.

Debido a que en la biblioteca de clases base hay muchas clases, se utiliza para identificarlas un mecanismo llamado espacio de nombres (namespace). La parte del nombre de la clase que se encuentra a la derecha del último punto se llama tipo de la clase. Todo lo que resta se llama espacio de nombres. Por ejemplo: En la clase llamada System.Runtime.InteropServices, InteropServices es el tipo de la clase y System.Runtime es el espacio de nombre. El espacio de nombre es una manera de organizar en grupos las distintas clases. Esto hace más manejable y fácil su uso.

La biblioteca de clases base es única e independiente del lenguaje. Permite el uso y la depuración de otros lenguajes. Es extensible ya que por el mecanismo de herencia el usuario puede crear nuevas clases que usan las clases base como "ladrillos". También el usuario puede incorporarlas en bibliotecas para su utilización posterior. Es segura ya que es posible permitir o restringir su uso por medio de distintos mecanismos de seguridad.

Los espacios de nombres más utilizados son los siguientes:

System: Es el espacio de nombres raíz para los tipos de datos en el framework .net. En particular, contiene la definición de la clase Object, que es el inicio de todos los tipos de datos del framework .net.

System.Windows: Contiene el conjunto de los elementos que permiten la creación de interfaces para usuarios de Windows.

System.Web: Contiene todos los recursos necesarios para la creación de aplicaciones web, con las clases de la tecnología ASP.NET, por ejemplo, o las clases utilizables para la creación de servicios web XML.

System.data: Presenta un conjunto de clases especializadas en el acceso a las bases de datos con el soporte de ADO.NET.

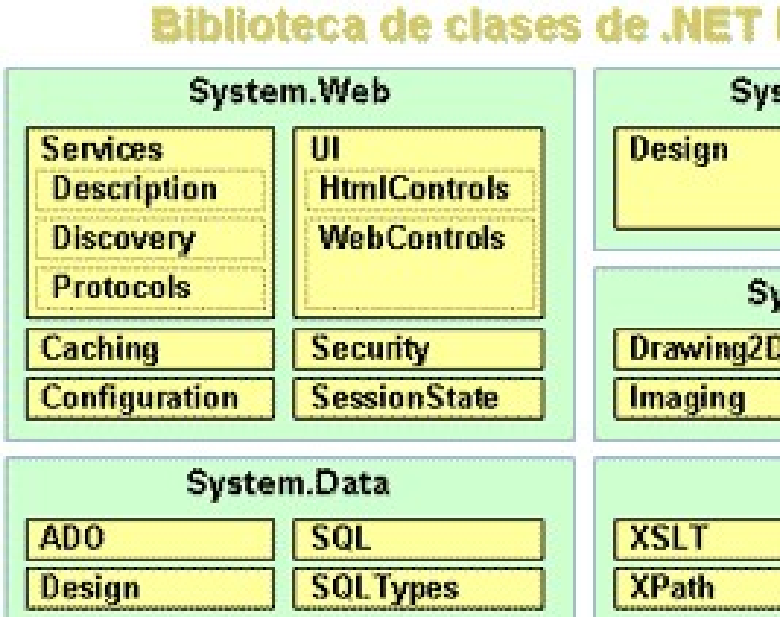
System.xml: Contiene las clases que permiten la manipulación de documentos XML.

System.Drawing: Contiene las clases para proporcionar toda la funcionalidad gráfica en el framework.

System.IO: Contiene las clases y los métodos necesarios para leer y escribir todo tipo de archivos, tanto en texto plano como a nivel de bytes.

Systems.Collections: Contiene una serie de clases que permiten administrar un conjunto de objetos de manera ordenada.

System.Security: Proporciona toda la funcionalidad para realizar los procesos de autenticación y autorización, manejo de credenciales y criptografía, entre otras tareas.



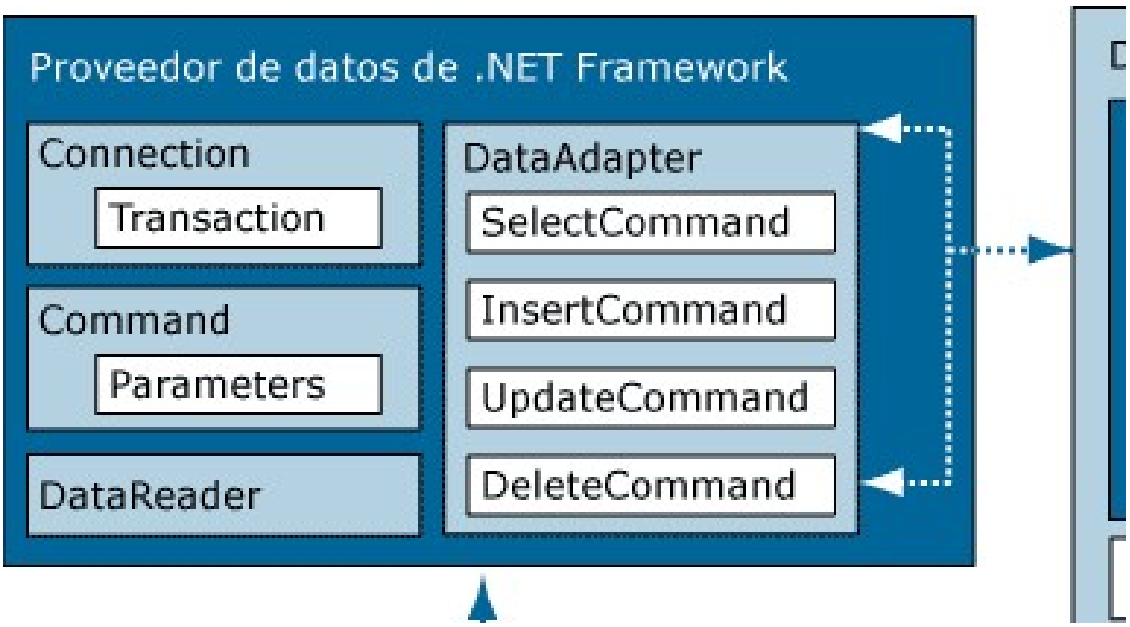
2.1.4 Acceso a datos

Tradicionalmente, el procesamiento de datos ha dependido principalmente de un modelo de dos niveles basado en una conexión. A medida que el procesamiento de datos utiliza cada vez más arquitecturas de varios niveles, los programadores están pasando a un enfoque sin conexión con el fin de proporcionar una escalabilidad mejor para sus aplicaciones.

ADO.NET aprovecha la eficacia de XML para proporcionar acceso a datos sin mantener una conexión abierta. ADO.NET fue diseñado teniendo en cuenta las clases de XML incluidas en .NET Framework: ambos son componentes de una única arquitectura.

ADO.NET y las clases de XML incluidas en .NET Framework convergen en el objeto **DataSet**. El DataSet se puede llenar con datos procedentes de un origen XML, ya sea éste un archivo o una secuencia XML.

En el siguiente diagrama se ilustran los componentes de la arquitectura de ADO.NET.



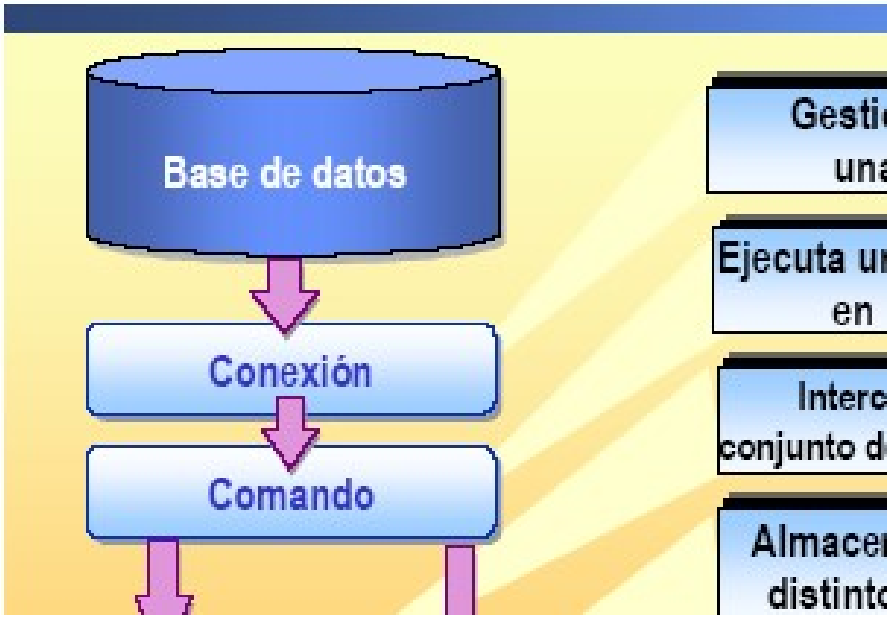
Los componentes de ADO.NET están diseñados para separar el acceso a datos de la manipulación de datos. ADO.NET tiene dos componentes principales que cumplen esta función: el DataSet y el **proveedor de datos de .NET Framework**, que es un conjunto de componentes entre los que se incluyen los objetos Connection, Command, DataReader y DataAdapter.

El DataSet de ADO.NET es el componente central de la arquitectura sin conexión de ADO.NET. El DataSet está expresamente diseñado para el acceso a datos independientemente del origen de datos. El DataSet contiene una colección de uno o más objetos DataTable formados por filas y columnas de datos, así como información sobre claves principales, claves externas, restricciones y relaciones relativa a los datos incluidos en los objetos DataTable.

El otro elemento central de la arquitectura de ADO.NET es el proveedor de datos de .NET Framework, cuyos componentes están diseñados expresamente para la manipulación de datos y para el acceso rápido a datos de avance de sólo lectura.

- ✓ El objeto **Connection** proporciona conectividad con un origen de datos.

- ✓ El objeto **Command** permite tener acceso a comandos de base de datos para devolver datos, modificar datos, ejecutar procedimientos almacenados y enviar o recuperar información sobre parámetros.
- ✓ El objeto **DataReader** proporciona una secuencia de datos de alto rendimiento desde el origen de datos.
- ✓ Por último, el objeto **DataAdapter** proporciona el puente entre el objeto DataSet y el origen de datos. El DataAdapter utiliza objetos Command para ejecutar comandos SQL en el origen de datos tanto para cargar el DataSet con datos como para reconciliar en el origen de datos los cambios aplicados a los datos incluidos en el DataSet.



Es posible escribir proveedores de datos de .NET Framework para cualquier origen de datos. La primera versión de .NET Framework incluía dos proveedores de datos de .NET Framework: el proveedor de datos de .NET Framework para OLE DB y el proveedor de datos de .NET Framework para SQL Server.

La versión actual del framework incluye cuatro proveedores de datos disponibles:

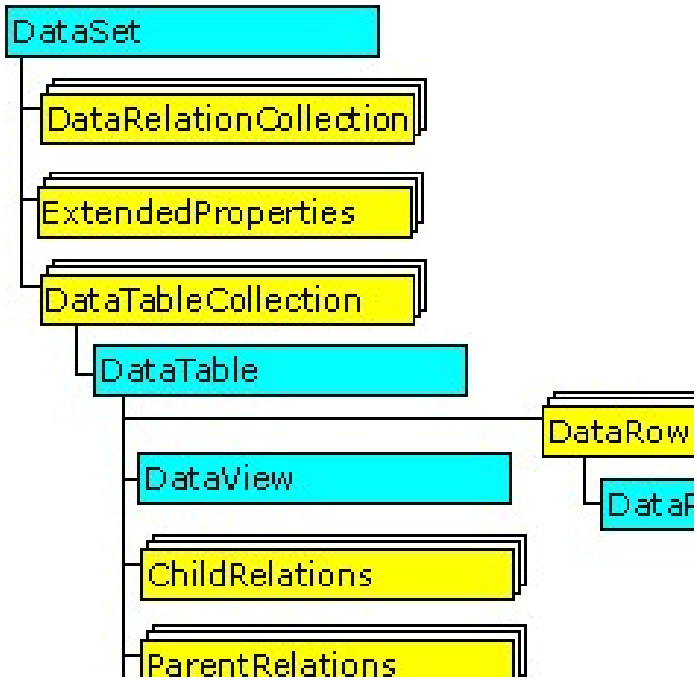
El proveedor para SQL Server: El proveedor de datos para SQL server utiliza un protocolo nativo para dialogar con el servidor de base de datos. Además, consume pocos recursos porque accede al servidor sin utilizar capas lógicas adicionales. Se puede utilizar con SQL server a partir de la versión 7. Todas las clases de este proveedor de datos están disponibles en el espacio de nombres System.Data.SqlClient.

El proveedor para OLE DB: El proveedor OLE DB utiliza la capa lógica OLE DB para comunicarse con el servidor de base de datos. Todas las clases están disponibles en el espacio de nombres System.Data.OleDb.

EL proveedor para ODBC: El proveedor ODBC utiliza un controlador ODBC nativo para comunicarse con el servidor de base de datos. Todas las clases están disponibles en el espacio de nombres System.Data.Odbc.

El proveedor para Oracle: Permite la conexión a una fuente de datos Oracle a través de las herramientas cliente de Oracle. Estas herramientas deben instalarse en el sistema operativo para poderse conectar a una base de datos Oracle. Es necesaria la versión 8.1.7 o una superior. Las clases están en el espacio de nombres System.Data.OracleClient.

El objeto DataSet es esencial para admitir escenarios de datos distribuidos de ADO.NET sin mantener una conexión. El DataSet es una representación residente en memoria de datos que proporciona un modelo de programación relacional coherente independientemente del origen de datos. Se puede utilizar con múltiples y distintos orígenes de datos, con datos XML o para administrar datos locales de la aplicación. El DataSet representa un conjunto completo de datos entre los que se incluyen tablas relacionadas, restricciones y relaciones entre las tablas. En la siguiente ilustración se muestra el modelo de objeto DataSet.



Un DataSet de ADO.NET contiene una colección de cero o más tablas representadas por objetos DataTable. La DataTableCollection contiene todos los objetos DataTable de un DataSet.

Un DataTable se define en el espacio de nombres System.Data y representa una única tabla de datos residentes en memoria. Contiene una colección de columnas representadas por una DataColumnCollection y restricciones representadas por una ConstraintCollection que, juntas, definen el esquema de la tabla. Un DataTable también contiene una colección de filas representadas por la DataRowCollection, que contiene los datos de la tabla. Junto con su estado actual, un DataRow conserva tanto la versión original como la actual para identificar los cambios realizados en los valores almacenados en la fila.

Un DataSet contiene relaciones en su objeto DataRelationCollection. Una relación, representada por el objeto DataRelation, asocia las filas de un DataTable con las filas de otro DataTable. Es análogo a una ruta de unión que podría existir entre las columnas de claves externas y principales en una base de datos relacional. Un DataRelation identifica columnas coincidentes en dos tablas de un DataSet.

Las relaciones permiten pasar de una tabla a otra dentro de un mismo DataSet. Los elementos esenciales de un DataRelation son el nombre de la relación, el nombre de las tablas relacionadas y las columnas relacionadas de cada tabla. Se pueden establecer relaciones con más de una columna por tabla, para lo que debe especificar una selección de objetos DataColumn como columnas clave. Cuando se agrega una relación al DataRelationCollection, se puede agregar también un UniqueKeyConstraint y un ForeignKeyConstraint para imponer restricciones de integridad cuando se realicen cambios en los valores de las columnas relacionadas.

El DataSet (así como el DataTable y el DataColumn) tiene una propiedad ExtendedProperties. ExtendedProperties es un PropertyCollection en el que se puede colocar información personalizada, como la instrucción SELECT que sirve para generar el conjunto de resultados, o una marca de fecha y hora de cuándo se generaron los datos. La colección ExtendedProperties se conserva con la información del esquema del DataSet (así como el DataTable y el DataColumn).

2.1.5 ASP.NET

ASP.NET proporciona un modelo de desarrollo Web unificado que incluye los servicios necesarios para crear aplicaciones Web para la empresa. Si bien ASP.NET es en gran medida compatible con la sintaxis de las páginas de Active Server (ASP), proporciona un modelo y una infraestructura de programación nuevos que permiten crear una eficaz clase de aplicaciones. ASP.NET forma parte de .NET Framework y permite aprovechar al máximo las características de Common Language Runtime, como la seguridad de tipos, la herencia, la interoperabilidad de lenguajes y las versiones.

ASP.NET es un entorno compilado basado en .NET. Se pueden crear aplicaciones en cualquier lenguaje compatible con .NET, como Visual Basic .NET, C# y JScript .NET. Además, .NET Framework está disponible en su totalidad para cualquier aplicación ASP.NET. Los programadores pueden aprovechar fácilmente las ventajas de estas

tecnologías, que incluyen el entorno Common Language Runtime administrado, seguridad de tipos, herencia, etc.

ASP.NET se ha diseñado para funcionar sin problemas con editores HTML WYSIWYG y otras herramientas de programación como Microsoft Visual Studio .NET. Todo esto, además de hacer más fácil la programación Web, ofrece todas las ventajas de estas herramientas, con una GUI que los programadores puede utilizar para ubicar controles de servidor en una página Web e integrar completamente la compatibilidad con la depuración.

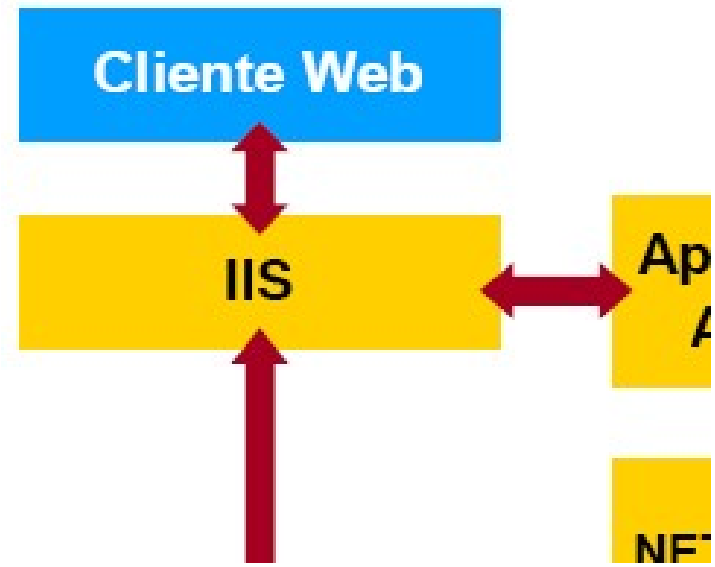
A la hora de crear una aplicación ASP.NET, los programadores pueden utilizar formularios Web Forms o servicios Web XML o combinarlas de la manera que más les convenga. Las dos características son compatibles con la misma infraestructura, que permite utilizar esquemas de autenticación, almacenar en caché datos que se utilizan con frecuencia y personalizar la configuración de la aplicación, entre otras muchas cosas.

Por otra parte, las aplicaciones Windows pueden proporcionar interfaces de usuario más potentes que nunca utilizando los espacios de nombres System.Windows.Forms y empleando los nuevos formularios Windows Forms de .NET.

Se puede utilizar las clases del espacio de nombres System.Windows.Forms para crear el IU cliente. Esta clase permite implementar el IU de Windows estándar en las aplicaciones .NET. Muchas funciones a las que anteriormente sólo podían accederse mediante llamadas a la API (Application Programming Interface) están ahora disponibles como parte de los propios formularios, lo cual hace el desarrollo mucho más fácil y potente.

Además, con la filosofía actual del framework, es posible separar en ficheros distintos la parte correspondiente a la presentación de la parte de codificación.

La arquitectura típica en el acceso a páginas de un proyecto ASP.NET es la expuesta a continuación:



2.1.5.1 Windows forms frente a Web forms

Cuando se diseñan aplicaciones que necesitan de un interfaz de usuario, existen dos opciones: Windows Forms y Web Forms. Ambos disponen de soporte completo en tiempo de diseño por parte del entorno de desarrollo y pueden generar un rico interfaz de usuario y funcionalidad avanzada de aplicaciones para solucionar problemas de negocio. Cuando se dispone de varias opciones, es importante saber qué opción utilizar en cada momento.

Windows Forms se utiliza para desarrollar aplicaciones en las que se espera que el cliente tenga una parte importante de la carga de proceso de una aplicación. Entre éstas se incluyen las aplicaciones clásicas de escritorio para el interfaz de programación de aplicaciones Microsoft Win32®. Algunos de los ejemplos incluyen aplicaciones de dibujo o gráficas, sistemas de introducción de datos, sistemas punto de venta, y juegos. Todas estas aplicaciones dependen de la potencia del equipo de sobremesa para procesar y mostrar contenido con alto rendimiento.

Web Forms de ASP.NET se utilizan para crear aplicaciones en las que el interfaz de usuario principal es un navegador. Incluyen aplicaciones pensadas para estar disponibles públicamente en la World Wide Web, como las aplicaciones de correo electrónico.



Característica	Windows Forms	
Implantación	Puede ejecutarse sin alterar el Registro	Nc
Gráficos	Incluye GDI+	Lo dir vu ac
Respuesta	Velocidad de respuesta más rápida posible para aplicaciones interactivas	Pu HT na
Plataforma	Requiere el .NET Framework ejecutándose	Só na

2.1.6 Windows Presentation Foundation

Windows Presentation Foundation (WPF) es un sistema de presentación de nueva generación para crear aplicaciones cliente de Windows que proporcionen una experiencia visual impactante para el usuario. Con WPF, se pueden crear una amplia gama de aplicaciones independientes, separando la presentación de la aplicación de la lógica.

El núcleo de WPF es un motor de representación basado en vectores e independiente de la resolución que se crea para sacar partido del hardware de gráficos moderno. WPF extiende el núcleo con un conjunto completo de características de desarrollo de aplicaciones que incluye Extensible Application Markup Language (XAML), controles, enlace de datos, diseño, gráficos 2-D y 3-D, animación, estilos, plantillas, documentos, multimedia, texto y tipografía. WPF se incluye en Microsoft .NET Framework, de modo que es posible compilar aplicaciones que incorporen otros elementos de la biblioteca de clases de .NET Framework.

WPF constituye un subconjunto de tipos de .NET Framework en su mayoría ubicados en el espacio de nombres System.Windows.

WPF proporciona mejoras de programación adicionales para el desarrollo de aplicaciones cliente de Windows. Una mejora evidente es la capacidad para programar una aplicación mediante código de lenguaje marcado y subyacente. En general, se utiliza el lenguaje marcado Extensible Application Markup Language (XAML) para implementar la apariencia de una aplicación, y los lenguajes de programación

administrados (subyacentes) para implementar su comportamiento. Esta separación entre la apariencia y el comportamiento aporta las ventajas siguientes:

- Se reducen los costos de programación y mantenimiento, al no estar el marcado específico de la apariencia estrechamente relacionado con el código específico del comportamiento.
- La programación es más eficaz porque los diseñadores pueden implementar la apariencia de una aplicación al mismo tiempo que los programadores implementan su comportamiento.
- Se pueden usar varias herramientas de diseño para implementar y compartir el marcado XAML, a fin de responder a los requisitos de quienes colaboran en la programación de aplicaciones.

XAML es un lenguaje de marcado basado en XML que se utiliza para implementar la apariencia de una aplicación mediante declaración. Se suele utilizar para crear ventanas, cuadros de diálogo, páginas y controles de usuario, así como para rellenarlos con controles, formas y gráficos.

XAML es un lenguaje declarativo de marcado. Tal y como se aplica en el modelo de programación .NET Framework, XAML simplifica la creación de la interfaz de usuario para una aplicación .NET Framework. Se pueden crear elementos visibles de la interfaz de usuario en el marcado XAML declarativo y, a continuación, separar la definición de la interfaz de usuario de la lógica en tiempo de ejecución mediante archivos de código subyacente, que se unen al marcado mediante definiciones de clases parciales. XAML representa directamente la creación de instancias de objetos en un conjunto concreto de tipos de respaldo definidos en ensamblados. Esto no es lo que sucede con la mayoría de lenguajes de marcado, que normalmente se interpretan sin esa relación directa con un sistema de tipos de respaldo. El código XAML habilita un flujo de trabajo en el que las partes independientes pueden funcionar en la interfaz de usuario y la lógica de una aplicación, a través de herramientas potencialmente diferentes.

2.1.7 Windows Communication Foundation

Windows Communication Foundation (WCF) es un marco de trabajo para la creación de aplicaciones orientadas a servicios. Con WCF, es posible enviar datos como mensajes asíncronos de un extremo de servicio a otro. Un extremo de servicio puede formar parte de un servicio disponible continuamente hospedado por IIS, o puede ser un servicio hospedado en una aplicación. Un extremo puede ser un cliente de un servicio que solicita datos de un extremo de servicio. Los mensajes pueden ser tan simples como un carácter o una palabra enviados como XML, o tan complejos como un flujo de datos binarios.

WCF incluye el siguiente conjunto de características:

Orientación a servicios: Como consecuencia del uso de los estándares de WS, WCF permite crear aplicaciones orientadas a servicios. SOA, la arquitectura orientada a

servicios, es el uso de servicios web para enviar y recibir datos. Los servicios tienen la ventaja general de estar débilmente acoplados entre una aplicación y otra. Una relación de acoplamiento débil implica que cualquier cliente creado en cualquier plataforma puede conectar con cualquier servicio siempre y cuando se cumplan los contratos esenciales.

Interoperabilidad: WCF implementa los estándares del sector modernos para la interoperabilidad de servicios web.

Varios patrones de mensajes: Los mensajes se intercambian mediante uno de los distintos patrones. El más común es el de solicitud/respuesta, en que un extremo solicita datos de otro extremo, y el otro extremo responde. Existen otros patrones, como un mensaje unidireccional, en que un único extremo envía un mensaje sin esperar ninguna respuesta.

Metadatos de servicios: WCF admite la publicación de metadatos de servicios utilizando formatos típicos como WSDL, Esquemas XML y WS-Policy. Estos metadatos pueden utilizarse para generar y configurar automáticamente clientes para el acceso a los servicios de WCF.

Contratos de datos: Dado que WCF se basa en .NET Framework, también incluye métodos con código sencillo para proporcionar los contratos que desea aplicar. Uno de los tipos de contrato universales es el contrato de datos. Básicamente, mientras se escribe el código del servicio usando Visual C# o Visual Basic, la forma más sencilla de controlar los datos consiste en crear clases que representan una entidad de datos con propiedades que pertenecen a la misma. WCF incluye un completo sistema para trabajar con datos de esta manera fácil. Cuando se han creado las clases que representan los datos, el servicio genera automáticamente los metadatos que permiten a los clientes ajustarse a los tipos de datos que se han diseñado.

Seguridad: Es posible cifrar los mensajes para proteger la privacidad, así como obligar a los usuarios a que se autentiquen antes de permitirles recibir mensajes. La seguridad puede implementarse utilizando estándares conocidos como SSL o WS-SecureConversation.

Varios transportes y codificaciones: Los mensajes pueden enviarse con cualquiera de los protocolos y codificaciones integrados. La combinación más frecuente de protocolo y codificación consiste en enviar mensajes SOAP codificados de texto sobre HTTP.

2.1.8 LINQ

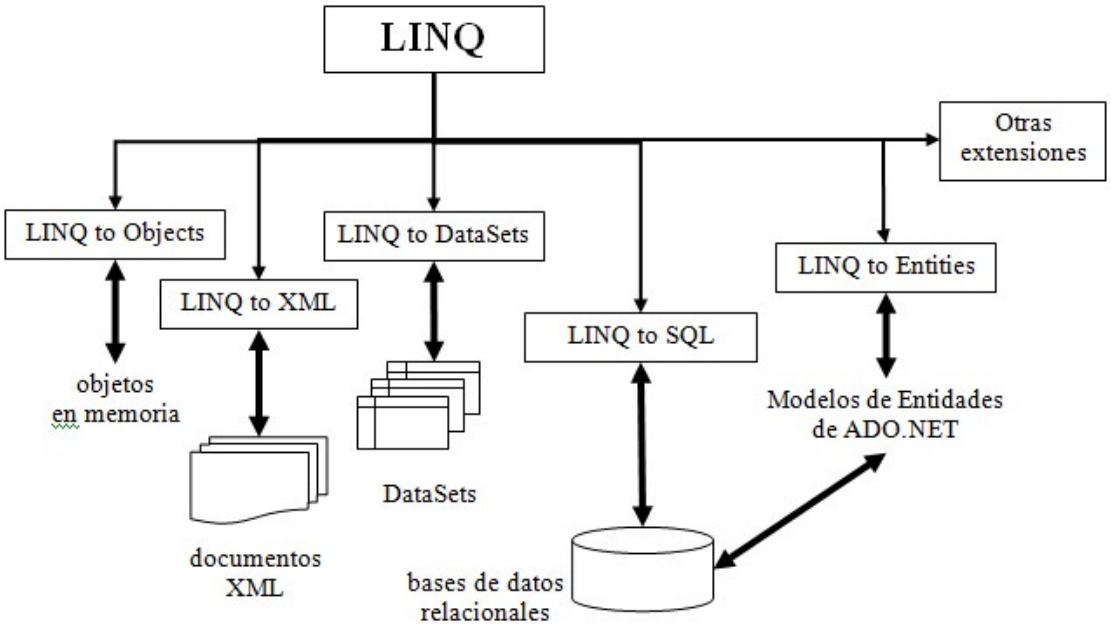
LINQ (Language-Integrated Query) es una combinación de extensiones al lenguaje de programación y librerías de código manejado que permite expresar de manera uniforme las consultas sobre colecciones de datos provenientes de distintas fuentes: objetos en memoria, documentos XML o bases de datos relacionales, entre otros.

El propio significado del acrónimo (consultas integradas en el lenguaje) ofrece la clave para comprender el objetivo principal de LINQ: permitir que el programador exprese las consultas sobre datos de diversa procedencia utilizando recursos de su propio lenguaje de programación. Las ventajas fundamentales de esta posibilidad son las siguientes:

- A la hora de expresar las consultas, el programador puede aprovechar todas las facilidades que el compilador y el entorno integrado ofrecen (verificación de sintaxis y de tipos por parte del compilador, ayuda IntelliSense dentro del entorno)
- LINQ permitirá, si no eliminar completamente, sí reducir en gran medida el ya mencionado fenómeno del "desajuste de impedancias" que provocan las diferencias entre los modelos de programación que proponen los lenguajes de propósito general y los lenguajes de consulta sobre bases de datos relacionales (SQL), documentos XML (XPath/XQuery) y otros almacenes de datos.
- Por último, otra ventaja importante de LINQ es que permitirá elevar el nivel de abstracción y claridad de la programación de las consultas. Por ejemplo, un programador que necesite hoy en día acceder a una base de datos, deberá especificar en un plan meticuloso cómo recuperar los datos que necesita; las expresiones de consulta, en cambio, son una herramienta mucho más declarativa, que permite en gran medida indicar únicamente qué se desea obtener, dejando al motor de evaluación de expresiones los detalles sobre cómo lograr ese objetivo.

El otro elemento clave es la arquitectura abierta de LINQ. Existen mecanismos estándar para aplicar LINQ a arrays y colecciones genéricas (LINQ to Objects), para ejecutar consultas integradas sobre documentos XML (LINQ to XML), para la ejecución de consultas en datos y rellenar DataSets (LINQ to DataSets), y para hacer consultas en bases de datos relacionales (LINQ to SQL y LINQ to Entities). No obstante, es posible la existencia de tecnologías que simplifican y uniformizan el acceso a otras fuentes de datos, como por ejemplo LINQ to Amazon y LINQ to SharePoint.

La arquitectura de LINQ puede ser descrita gráficamente de la siguiente forma:



En realidad, LINQ funciona como una capa intermedia entre la fuente de datos y el lenguaje que se está utilizando. Desde una perspectiva de desarrollador, es simplemente una nueva manera de consultar datos desde diversas estructuras de datos directamente en el IDE. En resumen, LINQ es un lenguaje de consulta que sirve de puente entre el lenguaje que se está utilizando y la fuente de datos de donde se quiere obtener los datos.

En la siguiente figura se puede ver el flujo de LINQ para llevar a cabo su funcionamiento:



La operación de LINQ inicia en el lenguaje que utiliza el desarrollador, como puede ser C# o Visual Basic. Para escribir las consultas se utilizan los operadores de LINQ y las extensiones del lenguaje.

Los operadores del lenguaje se exponen a través del API de operadores de consulta estándar. Entre los operadores más comunes tenemos:

- Select / SelectMany
- Where
- Sum / Min / Max / Average / Aggregate
- Join / GroupJoin
- GroupBy
- OrderBy / ThenBy

Las extensiones del lenguaje son implementadas de manera opcional por los lenguajes que implementan LINQ. Estas extensiones incluyen:

- Expresiones Lambda
- Inicializadores de Objetos
- Tipos Anónimos
- Sintaxis de Consulta
- Variables tipificadas implícitamente

Seguidamente el motor de LINQ se encarga de transformar estas consultas en un conjunto de llamadas a diversos métodos para obtener los datos vía los providers de LINQ.

El proveedor de LINQ es el que permite llevar a cabo la consulta desde el lenguaje hacia la fuente de datos seleccionada. Este proveedor de datos debe de implementar las interfaces IQueryProvider e IQueryable contenidas en el espacio de nombres System.Linq. Con estas interfaces se podría extender la funcionalidad de LINQ para realizar consultas a fuentes de datos que nosotros mismos seleccionamos, por ejemplo, servicios web, bases de datos no soportadas aún, etc.

La siguiente figura nos muestra la arquitectura de LINQ en detalle:



2.1.9 Herrramienta Desarrollo: MS Visual Studio.NET

Como herramienta de desarrollo del framework se encuentra MS Visual Studio .NET. Visual Studio .NET es un conjunto de aplicaciones completo para la creación tanto de

aplicaciones de escritorio como de aplicaciones Web de empresa para trabajo en equipo. Aparte de generar aplicaciones de escritorio de alto rendimiento, se pueden utilizar las eficaces herramientas de desarrollo basado en componentes y otras tecnologías de Visual Studio para simplificar el diseño, desarrollo e implementación en equipo de soluciones para empresa.

Visual Studio .NET es un conjunto completo de herramientas de desarrollo para la construcción de aplicaciones Web ASP, servicios Web XML, aplicaciones para escritorio y aplicaciones móviles. Visual Basic .NET, Visual C++ .NET, Visual C# .NET y Visual J# .NET utilizan el mismo entorno de desarrollo integrado (IDE), que les permite compartir herramientas y facilita la creación de soluciones en varios lenguajes. Asimismo, dichos lenguajes aprovechan las funciones de .NET Framework, que ofrece acceso a tecnologías clave para simplificar el desarrollo de aplicaciones Web ASP y servicios Web XML.

Entre sus características destacan las siguientes:

- ✓ Única herramienta de desarrollo para todo
- ✓ Soporta múltiples lenguajes de .NET: Visual Basic.NET, C#, Visual C++.NET, etc.
- ✓ Plataforma completa para creación, pruebas e integración de Servicios Web
- ✓ Plataforma RAD de alta productividad:
    - o Diseñadores visuales de Formularios (WebForms y WinForms)
    - o Diseñadores visuales de Componentes
    - o Depuración extremo a extremo, a través de lenguajes
    - o Tecnología IntelliSense y Ayuda Dinámica
- ✓ Extensible y Abierto

2.1.10 Versiones de la plataforma .net

La primera versión de la plataforma .NET se liberó en enero de 2002. Esta versión se sustituyó rápidamente por la versión 1.1, que corrigió algunos errores de la versión anterior y añadió algunas tecnologías que sólo estaban disponibles anteriormente como instalaciones independientes.

Las principales aportaciones de la versión 1.1 fueron las siguientes:

- Los proveedores de datos para ODBC y para Oracle, que inicialmente sólo estaban disponibles a través de descarga, se suministraron con el framework .net directamente.
- Se ofrece compatibilidad con la versión 6 del protocolo IP.

La versión 2.0 del framework introdujo, entre otras, las siguientes novedades:

- Compatibilidad con la nueva generación de ordenadores de 64 bits.

- Una evolución mayor en el acceso a las bases de datos con ADO.NET 2.0, mejorando el uso de XML.
- Una nueva versión de ASP.NET, que permite mayor facilidad en el desarrollo de aplicaciones web, incluyendo nuevos controles.

La versión 3.0 introduce nuevas tecnologías, destacando las siguientes:

- Windows Presentation Foundation (WPF): Representa un nuevo sistema de interfaces gráficas. Se basa en un motor de aspecto vectorial y permite una separación más clara entre la definición de la interfaz gráfica de una aplicación y su código, lo que permite repartir más fácilmente las tareas entre desarrolladores y diseñadores. Para ello, hace uso del lenguaje XAML (eXtensible Application Markup Language).
- Windows Communication Foundation (WCF): Constituye la nueva base de desarrollo de aplicaciones distribuidas. Facilita la comunicación entre aplicaciones añadiendo una capa de abstracción que uniformiza las técnicas de comunicación entre estas aplicaciones.
- Windows Workflow Foundation (WF): Está compuesto por una plantilla de programación, un motor de ejecución y herramientas para integrar flujos de trabajo en una aplicación. Se puede definir un flujo de trabajo como un conjunto de acciones o etapas que se ejecutan en un orden predefinido. Estas acciones se pueden encadenar en función de condiciones, interacciones con procesos informáticos o en función de interacciones humanas.
- Windows Cardspace facilita una nueva técnica a los usuarios para identificarse en una aplicación. Tiene la misma vocación que Microsoft Passport, pero no es específico de las aplicaciones Microsoft.

La versión 3.5 aporta ciertas evoluciones a la versión anterior. La gran novedad de esta versión es la aparición del lenguaje LINQ (Language Integrated Query). Este lenguaje permite uniformizar el método utilizado para extraer información de una fuente de datos. Este nuevo lenguaje permite consultar colecciones de objetos, bases de datos SQL server, DataSet de ADO.NET y documentos XML con la misma sintaxis.

La versión 4.0 mejora las capacidades y funciones disponibles. Entre las novedades más importantes se pueden destacar las siguientes:

- Se introducen mejoras en el recolector de basura, permitiendo mejorar el rendimiento de la aplicación.
- Se introducen nuevos tipos de datos, como BigInteger y Complex, que permiten el uso de valores numéricos enteros sin límite superior o inferior, y la manipulación de números complejos.
- Se mejoran las clases para manipular sistemas de archivos y crear archivos mapeados en memoria.
- Se pueden tener en cuenta más fácilmente los procesadores multinúcleo que permiten el reparto de los procesos con la programación paralela.

La versión 4.5 añade ligeras mejoras en relación a la versión 4.0. Destaca fundamentalmente la simplificación de la programación en modo asíncrono. Esta técnica impide ciertos bloqueos que se producían anteriormente.

La versión 4.6 del framework presenta las siguientes novedades:

- Windows Presentation Foundation (WPF): Visual Studio está desarrollado ahora con Windows Presentation Foundation.
- RyuJIT: RyuJIT es el nombre de la siguiente generación de compiladores Just-In-Time (JIT) para .NET. Utiliza una nueva arquitectura de alto rendimiento, que permite conseguir un alto rendimiento de procesado durante la compilación del JIT.
- SIMD: Este juego de instrucciones (SIMD) que todos los procesadores incluyen, ahora se pueden utilizar desde .NET Framework para versiones de 64 bits del CLR.
- Actualizaciones del recolector de basura: El recolector de basura se ha actualizado para reducir la latencia y mejora la utilización de la memoria.
- Windows Communication Foundation (WCF): WCF soporta ahora versión de TLS 1.1 y 1.2, además de las existentes SSL 3.0 y TLS 1.0. Ahora es posible seleccionar qué protocolos se usarán, además de poder deshabilitar otros protocolos más antiguos e inseguros. Por defecto, la versión 4.6 utiliza TLS 1.2.
- Mejoras en ADO.NET: ADO.NET soporta la característica de SQL Server 2016 “Always Encypted” que permite cifrar los valores de ciertas columnas en SQL Server 2016.
- Rendimiento del cargador de ensamblados del CLR: El cargador de ensamblados utiliza ahora la memoria de forma más eficiente descargando imágenes IL (intermediate language) después de que la imagen nativa haya sido cargada.
- Actualizaciones del soporte de criptografía: Se han actualizado las APIs de criptografía de .NET Framework para dar soporte a las nuevas APIs de criptografía de Windows. De esta manera se soportan nuevos algoritmos de cifrado.
- Tiempo Unix: Ahora se puede convertir valores de tiempo que están almacenados en formato Unix de forma más cómoda.

La versión 4.7 del framework presenta las siguientes novedades:

- Windows Communication Foundation (WCF): Permite configurar TLS 1.2 como protocolo de seguridad de mensajes predeterminado.
- Supervisión de memoria: Se notifica a las aplicaciones cuando se ejecutan cerca del límite de bytes privado configurado para el proceso o cuando queda poca RAM física total disponible en la máquina.

3 Modelo de programación

3.1 Introducción

Actualmente, ASP.NET soporta tres modelos de programación: ASP.NET Web Forms, ASP.NET MVC y ASP.NET Web Pages. Aunque los tres modelos de programación se ejecutan sobre la misma base de ASP.NET, cada uno de ellos estructura la aplicación de manera completamente distinta, promueve metodologías de desarrollo diferentes y se adapta a perfiles de desarrolladores distintos.

Algunas características que son virtudes en unos modelos de programación pueden ser consideradas debilidades en el otro. ¿Qué es más importante, desarrollar a un gran nivel de abstracción o tener control total cada uno de los aspectos de la aplicación? Simplicidad vs. Control, flexibilidad vs. Eficiencia: Estas son las compensaciones que hay que evaluar a la hora de elegir.

Es importante recalcar que el hecho de elegir uno de los modelos de programación al comenzar un proyecto de ASP.NET no excluye necesariamente a los otros, sino que es posible tener aplicaciones "híbridas" y en muchos casos tendrá todo el sentido desarrollar ciertas partes de la aplicación con un modelo de programación y otras partes con otro modelo distinto.

- ASP.NET Web Forms: Fue el primero de los tres modelos de programación en existir, y proporciona un gran nivel de abstracción con un modelo de programación familiar basado en eventos y controles que favorece la productividad mediante la programación declarativa reduciendo la cantidad de código necesaria para implementar una determinada funcionalidad.
- ASP.NET MVC: Se concibió como alternativa a Web Forms y proporciona un modelo de programación basado en el popular patrón de arquitectura MVC. Entre sus principales características destacan su completa integración con pruebas unitarias y su separación más clara entre la lógica de presentación, la lógica de negocio y la lógica de acceso a datos.
- ASP.NET Web Pages: Es el más reciente de los tres modelos de programación, y fue creado como respuesta a una creciente demanda de desarrolladores web sin experiencia previa con ASP.NET, cuya iniciación en ASP.NET Web Forms o MVC les suponía una inversión inicial de tiempo demasiado grande. Web Pages proporciona un modelo de programación más simple y rápido de aprender, sin renunciar a toda la funcionalidad y flexibilidad de ASP.NET.

3.1.1 ASP.NET Web Forms

ASP.NET es uno de los pilares fundamentales del Framework .NET, y durante muchos años Web Forms fue el único modelo de programación disponible. Cuando nació, Web Forms introducía novedades importantes respecto a otras tecnologías de desarrollo web como ASP, PHP o JSP. Una de las más significativas era la separación entre el código de servidor y el markup HTML.

La estructura típica de una página Web Forms se compone por un lado de un fichero *.aspx en el que se define la estructura HTML y se añaden declarativamente los controles de servidor de la página. Por otro lado, está el fichero de código code behind, *.aspx.cs o *.aspx.vb dependiendo del lenguaje de programación elegido, en el que se implementa el código de servidor de la página. Posteriormente, ambos ficheros se compilan en ensamblados que contienen el código de servidor que genera el HTML emitido al ejecutar la página.

La unión entre ambos ficheros se realiza mediante la directiva de página del fichero *.aspx ("<%@ Page … %>") en la que se especifica el nombre de la clase de la que hereda ("Inherits") y el fichero de código fuente ("CodeBehind") donde se implementa dicha clase, también conocido como el code behind de la página.

En el fichero de code behind se implementa el código de servidor de la página como los manejadores de eventos de los controles de la página, código de interacción con las capas de lógica de negocio o acceso a datos, o código de validación de datos de entrada del usuario.

Web Forms es un modelo de programación cuya principal característica es el nivel de abstracción y encapsulamiento que proporciona, a la vez que es tremendamente versátil.

3.1.2 ASP.NET MVC

En marzo de 2009 se hizo pública la primera versión de ASP.NET MVC. El patrón de arquitectura MVC (model-view-controller) no es nuevo (data de 1979) ni es algo que haya inventado Microsoft. Existen muchos frameworks de desarrollo web populares que utilizan MVC, como por ejemplo Ruby on Rails, Spring o Apache Struts. MVC es un patrón de arquitectura que ayuda a crear una separación lógica entre el modelo (la lógica de acceso a datos), la vista (la lógica de presentación) y el controlador (la lógica de negocio).

Uno de los pilares básicos de ASP.NET MVC es el concepto de enrutamiento (routing), lo que permite a las aplicaciones aceptar peticiones a URLs que no se corresponden con ficheros físicos en el servidor. Por ejemplo, en ASP.NET Web Forms las URLs tienen el siguiente formato "http://website/products.aspx?category=dvd" en el que físicamente existe un fichero products.aspx en la raíz del sitio web. En MVC la misma URL tendría el siguiente aspecto "http://website/products/dvd" sin que el servidor web necesariamente contenga una carpeta products con una subcarpeta dvd. De forma predeterminada, ASP.NET MVC enruta las peticiones al controlador y a la vista adecuada en función de la URL. Es decir, en el ejemplo anterior, nos devolverá la vista dvd del controlador products.

El controlador es la pieza responsable de responder a las peticiones realizadas contra la aplicación ASP.NET MVC mediante distintas "acciones", por ejemplo, devolviendo una vista determinada, redireccionando la petición a un controlador distinto, etc. En el caso más básico, las acciones de un controlador se limitan a devolver vistas.

ASP.NET MVC fue concebido con el objetivo de proporcionar una mayor separación de conceptos y mayor control sobre el comportamiento de la aplicación. Al no hacer uso

de muchos de los mecanismos de abstracción de ASP.NET Web Forms, como el view state o los formularios de servidor, ASP.NET MVC requiere un mayor conocimiento sobre protocolos y lenguajes web. En general las aplicaciones ASP.NET MVC requieren desarrollar una mayor cantidad de código que una aplicación ASP.NET Web Forms de funcionalidad equivalente, pero también proporcionan un mayor control al desarrollador sobre el comportamiento de la aplicación.

3.1.3 ASP.NET Web Pages

Transcurrido poco más de un año desde el lanzamiento de la primera versión de ASP.NET MVC, en Julio de 2010 se anunció la disponibilidad pública de una nueva sintaxis para páginas ASP.NET llamada "Razor" y un nuevo tipo de páginas ASP.NET que hacen uso de dicha sintaxis, las denominadas ASP.NET Web Pages. Prácticamente de forma simultánea al lanzamiento de Razor, se hizo pública la disponibilidad de Microsoft WebMatrix. WebMatrix es una herramienta de desarrollo web gratuita que incluye todo lo necesario para desarrollar aplicaciones ASP.NET, ASP o incluso PHP. WebMatrix proporciona un entorno de desarrollo web autónomo, que integra también el servidor web de desarrollo (IIS Express), la base de datos (SQL Server Compact Edition), y las herramientas para desplegar las aplicaciones en entornos de producción (Web Deploy).

La sintaxis Razor fue diseñada con los objetivos de ser fácil de aprender, minimizar el número de caracteres necesarios para la sintaxis consiguiendo un código compacto y fluido, utilizar los lenguajes .NET existentes, ser compatible con cualquier editor de texto y ser "testeable" mediante pruebas unitarias. En esencia, Razor permite introducir fragmentos de código Visual Basic .NET o C# directamente en el código HTML, por lo que convenientemente la extensión de las páginas Razor es *.cshtml o *.vbhtml.

Con Razor todo el código de una página se encuentra en un único fichero, que se ejecuta de principio a fin. El ciclo de vida de ejecución de una página se ha simplificado significativamente respecto a Web Forms o MVC. El comienzo de un bloque de código de servidor en Razor se indica con el carácter '@', y a diferencia de la sintaxis equivalente en las páginas *.aspx, '<% … %>', no se indica explícitamente el cierre del bloque. El compilador utiliza su conocimiento de los lenguajes .NET para determinar donde terminan los bloques de código de servidor, y donde comienza el HTML.

3.2 Lenguajes de programación

.NET Framework soporta múltiples lenguajes de programación y aunque cada lenguaje tiene sus características propias, es posible desarrollar cualquier tipo de aplicación con cualquiera de ellos.

Todos los lenguajes de programación soportan dos especificaciones:

Common Language Specification (CLS)

La especificación de lenguaje común (CLS) es un estándar donde se especifican todas las características que soporta el CLR, la mayoría necesarias para implementar cualquier aplicación. Indistintamente del lenguaje de programación en que sean escritos, todos los componentes desarrollados y compilados de acuerdo al CLS pueden interactuar entre sí. El .Net Framework incluye cuatro lenguajes de programación de alto nivel especificados de acuerdo al CLS:

• Microsoft Visual Basic .NET

• Microsoft Visual C# .NET

• Microsoft Visual J#.NET

• Microsoft Visual C++.NET

Esto permite por ejemplo que se pueda incrustar dentro de un programa escrito en Visual Basic.NET fragmentos escritos en C# o C++ .Net.

El estándar CLS está a disposición pública, esto ha permitido que diseñadores de otros lenguajes y compiladores diseñen lenguajes compatibles con la especificación y por ende compatibles entre sí y con los lenguajes de .Net.

Es importante destacar que la elección de un lenguaje específico para el desarrollo de una aplicación en .Net se ha convertido en una cuestión que depende directamente del gusto del desarrollador. Los aspectos de eficiencia del compilador no son tomados en cuenta dado que todos los compiladores generan los ensamblados de acuerdo al CLS y los ensamblados son ejecutados por el CLR sin tener en cuenta el lenguaje en que se desarrollaron.

Common Type System (CTS)

El sistema común de tipos de datos (CTS) brinda una definición común de los tipos de datos básicos que utiliza el CLR. Además, define la forma en que se declaran, usan y manejan en tiempo de ejecución los tipos de datos. El CTS permite que los lenguajes que compilan para una plataforma CLI (Common Language Infrastructure) compartan el mismo sistema de tipos de datos. De esta manera, se facilita la interoperabilidad entre lenguajes de programación.

4 Servicios de Azure

4.1 Introducción

Azure es un conjunto integral de servicios en la nube que los desarrolladores y los profesionales de TI utilizan para crear, implementar y administrar aplicaciones a través de una red global de centros de datos. Herramientas integradas, DevOps y un marketplace permiten crear de manera eficaz cualquier cosa, desde aplicaciones móviles sencillas hasta soluciones orientadas a Internet.

4.2 Hospedaje de aplicaciones

La primera decisión que se tendrá que tomar es dónde se debe hospedar la aplicación. Azure ofrece varias opciones de hospedaje.

4.2.1 Máquinas virtuales

Una de las formas de hospedar una aplicación es usar una máquina virtual en Azure Virtual Machines. Esto da mucho control sobre el modo de hospedar la aplicación, pero también obliga a mantener el entorno, por ejemplo, a aplicar revisiones al sistema operativo (SO) y mantener actualizados los programas antivirus.

Se puede, por ejemplo, usar una máquina virtual para probar la última versión de preview de Visual Studio sin "ensuciar" el equipo.

4.2.2 Contenedores

Los contenedores son mucho más ligeros que las máquinas virtuales y se pueden iniciar y detener en cuestión de segundos. Los contenedores ofrecen además una gran portabilidad que los hace idóneos para desarrollar una aplicación localmente en el equipo local y hospedarla después en el cloud, primero en la fase de prueba y más adelante en la producción. Se puede incluso ejecutar los contenedores onpremises o en otros clouds. El entorno que se usa en un equipo de desarrollo viaja con el contenedor, por lo que la aplicación se ejecuta siempre en el mismo ecosistema.

Al igual que las máquinas virtuales, los contenedores proporcionan mucho control sobre el entorno. Se puede instalar todo lo necesario para ejecutar las aplicaciones. Pero, de nuevo, será el programador el responsable de aplicar las revisiones y mantener el SO que se ejecuta en el contenedor, así como de otros elementos auxiliares como los programas antivirus.

4.2.3 Hospedaje de contenedores con Azure Container Instances

Se puede hospedar contenedores usando Azure Container Instances. Con este servicio, se puede integrar fácilmente un contenedor sin necesidad de un orquestador de contenedores, como Kubernetes, y sin tener que administrar los recursos que se hospedan en el contenedor.

4.2.4 Hospedaje de contenedores con Azure Container Service

Otra forma de hospedar contenedores es usar Azure Container Service. Con este servicio, puede escalar y administrar los contenedores usando orquestadores como Mesosphere DC/OS, Docker Swarm o Kubernetes. Este servicio es una forma excelente para empezar a mover los contenedores al cloud.

4.2.5 Azure Functions

Con Azure Functions, solo se tiene que escribir el código que se necesita para una solución sin preocuparse por crear una aplicación completa o la infraestructura para ejecutarla. Una función es una unidad de lógica de código desencadenada por una

solicitud HTTP, un evento de otro servicio de Azure o una programación. Los enlaces de entrada y de salida conectan el código de la función con otros servicios, como Azure Blob Storage, Azure Cosmos DB y Azure Service Bus, con una cantidad mínima de código. Con Functions, se puede crear funciones con rapidez y hospedarlas en un entorno elástico que realiza el escalado de forma automática.

Se puede escribir funciones de Azure en C#, F#, Node.js, Java, PHP y en una lista creciente de lenguajes.

Un ejemplo de una aplicación que usa Functions es una que activa una función cada vez que se carga un nuevo archivo de imagen en Azure Blob Storage. Al hacerlo, la función ajustaría el tamaño de la imagen y la escribiría en otra cuenta de Blob Storage. La firma de la función de este ejemplo tendría un aspecto similar al siguiente (en una secuencia de comandos de C#):

public static void Run (Stream myBlob, string name, TraceWriter log, BlockBlob outputBlob) { }

Los datos de Blob que desencadenaron la función se pasan a la función como el parámetro myBlob, que incluye la dirección URL del Blob. Puede usar el parámetro de enlace de salida outputBlob para especificar el Blob en el que desea escribir el resultado. No es necesario escribir el código para conectar a Blob Storage, solo configurarlo.

4.2.6 Azure Logic Apps

Se puede organizar la lógica de negocios con Logic Apps, automatizar un proceso de negocio o integrar con aplicaciones de software como servicio (SaaS). Al igual que en el caso de Azure Functions, Logic Apps se puede activar desde una fuente externa; por ejemplo, un nuevo mensaje en una cola de Azure Storage. Puede enlazar llamadas de la API a conectores para crear un flujo de trabajo (posiblemente complejo) en el que participen recursos del cloud y on-premises. Logic Apps tiene disponibles muchos conectores con las API, como el que permite conectar con Azure SQL Databases, SalesForce, SAP, etc. También puede exponer sus propias API o funciones de Azure como conectores para usarlos en una aplicación lógica, lo que le permitirá realizar fácilmente acciones en sistemas externos en su flujo de trabajo o hacer que uno de ellos active su aplicación lógica.

Al igual que las funciones de Azure, las aplicaciones lógicas no tienen servidor, se escalan automáticamente y solo se paga por ellas cuando se ejecutan.

El siguiente ejemplo muestra un flujo de trabajo de Logic Apps:

1. La aplicación lógica se activa mediante un correo electrónico que contiene un pedido de envío que llega a Office 365.
2. Usando los datos del correo electrónico, la aplicación lógica comprueba la disponibilidad del artículo pedido en SQL Server.
3. La aplicación lógica envía un mensaje de texto al teléfono del cliente utilizando Twilio (el número de teléfono estaba también en el correo electrónico), y en él se indica que el pedido se ha recibido y el artículo se ha enviado

4.2.7 Azure App Service

Como alternativa, se pueden hospedar las aplicaciones en una de las principales ofertas de servicio de Azure: Azure App Service. Azure App Service es un conjunto de servicios de hosting y orquestación con características y capacidades comunes. Por ejemplo, todos los App Services tienen la capacidad de proteger una aplicación utilizando Azure Active Directory y pueden usar dominios personalizados.

Azure App Service consta de los siguientes elementos:

- Aplicación web: Aplicación web es uno de los servicios de Azure más usados. Puede utilizarse para hospedar las aplicaciones web o las API's. Una aplicación web es básicamente una abstracción de un servidor web, como Internet Information Services (IIS) o Tomcat, que se utiliza para hospedar las aplicaciones basadas en HTTP. Aplicación web puede hospedar aplicaciones que están escritas en .NET, Node.js, PHP, Java o Python, y existen extensiones que se pueden usar para ejecutar otros lenguajes.
- Web App for Containers: Web App for Containers ayuda a implementar y ejecutar fácilmente aplicaciones web con contenedores a escala. Se pueden obtener imágenes del contenedor de Docker Hub o de Azure Container Registry, y Web App for Containers. La plataforma se encarga automáticamente de las revisiones del SO, del aprovisionamiento de capacidad y del equilibrio de carga
- Aplicación móvil: Aplicación móvil proporciona un back-end para las aplicaciones móviles. Se puede hospedar en Aplicación móvil una API con la que se conectan las aplicaciones móviles a través del SDK cliente multiplataforma. Está disponible para iOS, Android, Windows y Xamarin para iOS y Android y Xamarin Forms. Aplicación móvil ofrece características únicas como la sincronización sin conexión y las notificaciones push que ayudan a crear una experiencia móvil segura, moderna y de alto rendimiento. Se puede escribir el back-end de Aplicación móvil en .NET o Node.js.

4.3 Azure Content Delivery Network

Uno de los servicios de Azure que pueden ayudar a hacer una aplicación más rápida es Azure Content Delivery Network. Se suben los archivos estáticos (vídeos, imágenes, JavaScript, CSS e incluso archivos estáticos HTML) a un almacén de datos como Azure Blob Storage y luego se asocia Azure Content Delivery Network. Content Delivery Network replicará a continuación esos archivos estáticos en cientos de puntos de presencia (PoP) repartidos por todo el mundo. Solo hay que cambiar la referencia a los archivos estáticos en la aplicación a una dirección URL diferente.

4.4 Azure Redis Cache

Todas las aplicaciones modernas trabajan con datos. Recuperar datos desde un almacén de datos, como una base de datos, conlleva normalmente explorar múltiples tablas o

documentos que se encuentran en algún servidor distante, combinar los resultados y enviar el resultado al dispositivo que inició la solicitud. Esto, por supuesto, lleva tiempo y puede frustrar y molestar a sus usuarios.

Para eliminar algunos de estos "recorridos", se puede almacenar en caché los datos que no cambian a menudo. De esta forma, en lugar de consultar la base de datos cada vez, se podrían recuperar algunos de los datos desde una caché, como Azure Redis Cache. La ventaja de la caché es que almacena los datos en un formato sencillo como clave-valor. No se necesita ejecutar una consulta compleja para obtener estos datos, basta con conocer la clave para recuperar el valor. Esto puede mejorar mucho el rendimiento de la aplicación. Así funciona este flujo de trabajo:

1. La aplicación necesita algunos datos e intenta recuperarlos de la caché.
2. Si los datos no están, los obtiene de la base de datos y los almacena también en la caché.
3. La próxima vez que la aplicación busque ese dato, lo encontrará en la memoria caché, lo que evitará un viaje a la base de datos.

Azure proporciona caché como servicio con Redis Cache. Esto se basa en el proyecto de código abierto Redis y cuenta ahora con el respaldo de los ingenieros y los SLA de Microsoft. Es muy eficaz e incluye opciones avanzadas como la agrupación en clústeres y la replicación geográfica.

4.5 Azure Traffic Manager

Muchas aplicaciones modernas tienen usuarios en todo el mundo. Proporcionar una experiencia de alto rendimiento para todo el mundo es, como poco, un reto. El problema más obvio al que enfrentarse es la latencia. La latencia es el tiempo que tarda una señal o una solicitud en viajar hasta un usuario. Cuanto más lejos esté el usuario de su aplicación, mayor será la latencia que experimente.

Azure Traffic Manager ajusta la escala entre distintas regiones, lo que ayuda a reducir la latencia y proporciona a los usuarios una experiencia superior, estén donde estén. Traffic Manager es un mecanismo de enrutamiento inteligente que se antepone a, por ejemplo, las aplicaciones de Aplicación web en todo el mundo. Aplicación web actúa como punto de conexión, y Azure Traffic Manager supervisa el estado y el rendimiento. Cuando un usuario accede a la aplicación, Traffic Manager le dirige a la aplicación de Aplicación web que es más eficaz en su proximidad.

Incluir Traffic Manager en la arquitectura es una buena forma de mejorar el rendimiento de las aplicaciones.

4.6 Azure SQL Database

Si se necesita una base de datos relacional para almacenar datos, Azure SQL Database es una buena opción. Es un sistema de base de datos relacional que es similar a Microsoft SQL Server on-premises. Como SQL Database se ejecuta en el cloud, está

totalmente administrado, ofrece un alto rendimiento y copias de seguridad automáticas, y es escalable, además de contar con muchas funciones avanzadas.

Se puede utilizar SQL Database con otras herramientas de Microsoft, como SQL Server Management Studio y Entity Framework.

A continuación, se describen algunas de las características más avanzadas:

- Replicación geográfica: que replica los datos en otras regiones geográficas en tiempo real
- Enmascaramiento dinámico de datos: enmascara dinámicamente los datos confidenciales para determinados usuarios en tiempo de ejecución
- Auditoría: proporciona una auditoría completa de todas las acciones que se realizan en los datos

Las bases de datos de SQL Database son extremadamente fiables y sólidas y ofrecen un acuerdo de nivel de servicio (SLA) que garantiza un tiempo de actividad del 99,99%.

4.7 Azure Database for MySQL

MySQL es un sistema de base de datos relacional de código abierto que utilizan millones de aplicaciones como WordPress, Joomla y Drupal. Ahora, MySQL está disponible como servicio de base de datos administrado en el servicio Azure Database for MySQL. Esto significa que se puede empezar a trabajar rápidamente sin necesidad de configurar o mantener servidores. Azure proporciona seguridad y escalabilidad "out of the box". Además, el servicio Database for MySQL ofrece la misma fiabilidad que SQL Database y proporciona un tiempo de actividad del 99,99%.

Como en el caso de SQL Database y Azure Database for PostgreSQL, se realizan copias de seguridad incrementales cada cinco minutos y copias de seguridad completas cada hora, lo que permite recuperar los datos en el estado que tenían hasta 35 días antes.

4.8 Azure Database for PostgreSQL

Azure Database for PostgreSQL, basado en la popular tecnología de base de datos PostgreSQL, proporciona un servicio de base de datos relacional de objeto. Esto es ligeramente diferente a los servicios de base de datos relacional, ya que las bases de datos relacionales de objeto pueden almacenar tipos de datos más complejos, como los tipos de datos anidados, y son por tanto más adecuadas para los lenguajes de programación orientada a objetos que se utilizan en muchas aplicaciones.

Database for PostgreSQL es un servicio administrado que tiene las mismas características que Database for MySQL. Se puede aumentar o reducir su escala, ofrece un alto nivel de disponibilidad (SLA del 99,99%) y crea copias de seguridad automáticas.

4.9 Azure Cosmos DB

Azure Cosmos DB es la nueva versión y la nueva marca de DocumentDB. Cosmos DB es un nuevo tipo de base de datos pensada realmente para el cloud. Estas son algunas de sus principales características:

- SLA del 99,99% que incluye bajas latencias (menos de 10 ms en las lecturas; menos de 15 ms en las escrituras).
- Replicación geográfica: replica los datos en otras regiones geográficas en tiempo real.
- Traffic Management: envía a los usuarios a la réplica de datos más próxima a ellos.
- Escala global sin límites: solo se paga por el rendimiento y el almacenamiento que se necesita.
- Indexación automática de los datos: ya no es necesario mantener ni ajustar la base de datos.

Además de todas estas características, Cosmos DB ofrece diferentes API con las que puede almacenar y recuperar datos, como SQL, JavaScript, Gremlin, MongoDB y Almacenamiento de tablas de Azure. Las distintas API gestionan los datos de diferentes formas. Se puede utilizar documentos como datos y también tablas no estructuradas, gráficos, blobs, etc. Se puede utilizar la API que se adapte a las necesidades. Rendimiento de nivel de cloud, escalabilidad y fiabilidad.

4.10 Azure Storage

Otra opción para almacenar datos es Azure Storage. Este es uno de los servicios más antiguos, eficaces y fiables de Azure. Azure Storage ofrece cinco tipos de almacenamiento que tienen las siguientes características comunes:

- Redundancia geográfica: replica los datos en distintos centros de datos para que puedan recuperarse si un centro de datos deja de funcionar en caso de desastre
- Cifrado de datos en tiempo de ejecución
- Dominios personalizados

Los cuatro tipos de Azure Storage son Blob, cola, archivo y disco.

4.10.1 Blob Storage

Se puede usar Azure Blob Storage para almacenar datos no estructurados de gran tamaño, lo que se conoce como "blobs". Pueden ser archivos de vídeo, imagen, audio o texto o incluso archivos de disco duro virtuales (VHD) para máquinas virtuales.

Hay dos tipos de blobs: blobs de página y de bloque. Los blobs de página están optimizados para las operaciones aleatorias de lectura y escritura. Son perfectos para el

almacenamiento de un disco duro virtual. Los blobs de bloques están optimizados para cargar eficientemente grandes cantidades de datos. Son idóneos para almacenar archivos de vídeo grandes que no cambian con frecuencia.

4.10.2 Queue Storage

Azure Queue Storage es un tipo poco usual de almacenamiento que se utiliza para almacenar pequeños mensajes de datos, pero su principal objetivo es servir como cola. Los mensajes se ponen en la cola y otros procesos los toman de ella. Este patrón desvincula el remitente del procesador de mensajes y mejora el rendimiento y la fiabilidad. Azure Queue Storage se basa en Microsoft Message Queueing, que se puede encontrar en versiones anteriores de Windows.

4.10.3 File Storage

Se puede usar Azure File Storage como una unidad desde la que se comparten los archivos. Usa el protocolo de bloque de mensaje de servidor (SMB), lo que significa que se puede utilizar con Windows y Linux, y se puede tener acceso a él desde el cloud o desde sistemas on-premises. Igual que los otros tipos de Azure Storage, File Storage es escalable y económico.

4.10.4 Disk Storage

Azure Disk Storage es similar a File Storage, pero está pensado específicamente para un alto rendimiento de E/S. Es perfecto para usarlo como unidad en una máquina virtual que necesite alto rendimiento, por ejemplo, una máquina virtual en la que se ejecute SQL Server. Sin embargo, hay que tener en cuenta que Disk Storage solo está disponible en el nivel de precios premium de Azure Storage.

4.11 Azure Data Lake Store

Los almacenes de datos anteriores estaban diseñados para el uso normal con aplicaciones o para el uso con máquinas virtuales. Azure Data Lake Store es un almacenamiento para aplicaciones de Big Data. Se usa para almacenar grandes cantidades de datos en su formato nativo: estructurados, no estructurados y cualquier tipo intermedio. La finalidad de Data Lake Store es albergar los datos sin procesar para poder analizarlos o transformarlos y moverlos. A continuación, se muestran las principales características de Azure Data Lake:

- Capacidad de almacenamiento ilimitada. Un archivo puede tener más de un petabyte.
- Rendimiento escalable para permitir grandes cantidades de análisis en paralelo.
- Puede almacenar datos en cualquier formato sin un esquema.

Este es un enfoque muy diferente al del almacén de datos tradicional, en el que los esquemas de datos se definen de antemano.

Por ejemplo, se podría utilizar Data Lake Store para almacenar todos los datos que se obtienen de los dispositivos del Internet de las cosas (IoT) que están recopilando datos de temperatura. Puede dejar los datos en el almacén y luego filtrarlos y crear una vista de los datos por hora o por semana.

4.12 Azure SQL Data Warehouse

Se puede usar Azure SQL Data Warehouse si se necesita una solución de almacenamiento de datos tradicional totalmente administrada, escalable en tamaño, segura y de alto rendimiento. Los datos se almacenan en esquemas predefinidos y se consultan usando el lenguaje SQL de SQL Server ya conocido.

Dado que SQL Data Warehouse se ejecuta en Azure, permite el uso de características avanzadas como la detección automática de amenazas, que utiliza Machine Learning para entender los patrones de la carga de trabajo y servir como sistema de alarma para avisar de posibles incumplimientos.

Un ejemplo de uso de SQL Data Warehouse es cuando se sabe qué informes se van a mostrar a los usuarios y cuál es el esquema de datos de esos informes. En ese caso, puede crear esquemas en SQL Data Warehouse y rellenarlos con los datos para que los usuarios pueden examinar los datos.

4.13 Búsqueda de Azure

La búsqueda es una característica habitual en la mayoría de las aplicaciones, y sin embargo siempre ha sido una función difícil de implementar. Búsqueda de Azure proporciona una buena parte de la base necesaria para hacer búsquedas. Solo hay que usar una instancia de Búsqueda de Azure, crear un índice que ayude a buscar y rellenarlo con datos. Hay muchas opciones que pueden usarse para adaptar Búsqueda de Azure y hay muchas características que facilitarán las búsquedas a los usuarios:

- Búsqueda geográfica que permite a los usuarios explorar datos en función de la proximidad de un resultado de búsqueda a una ubicación física.
- Los analizadores de lenguaje de Lucene, así como los procesadores de lenguaje natural de Microsoft (NLP), disponibles en 56 idiomas para gestionar de forma inteligente la lingüística de cada idioma, incluidos los tiempos verbales, los sustantivos con plurales irregulares (por ejemplo, "vez" frente a "veces"), la descomposición de palabras y la división de palabras.
- Supervisión y creación de informes que indican lo que se ha buscado, lo rápida que ha sido la búsqueda y si ha tenido éxito.
- Características de experiencia del usuario, como ordenar y paginar los resultados de búsqueda, filtrado inteligente y sugerencias de búsqueda.

Se podría implementar Búsqueda de Azure para ayudar a los usuarios a realizar búsquedas en un catálogo de productos en una aplicación de comercio electrónico.

4.14 Azure Cognitive Services

Con Azure Cognitive Services se pueden agregar funciones inteligentes a las aplicaciones. Hay más de veinticinco APIs en las categorías de visión, habla, lenguaje, conocimiento y búsqueda.

Entre las APIs disponibles se encuentran las siguientes:

- Con Face API se puede realizar reconocimiento e identificación facial e incluso determinar el color de ojos de una persona, aunque esa persona lleve gafas.
- Emotion API determina las emociones que una persona muestra en una foto.
- Language Understanding Intelligent Services API (o LUIS) puede entender texto a partir del modelo que se use para entrenarla.
- Translator Speech API convierte la voz en texto y lo traduce en tiempo real.

Cognitive Services proporciona estas funciones inteligentes a través de Machine Learning e inteligencia artificial, que se puede entrenar con los propios conjuntos de datos. Estos servicios abren posibilidades con las que no se podía siquiera soñar hace tan solo unos años.

4.15 Azure Service Bus

En Azure, la base de la mensajería es Azure Service Bus. El servicio Service Bus incluye una colección de servicios que se pueden utilizar para los patrones de mensajería. Entre ellos destacan Colas y Temas de Azure Services Bus.

Las colas de Azure Service Bus se usan para desacoplar sistemas. Por ejemplo: una aplicación web recibe pedidos de los usuarios y tiene que invocar un servicio web para procesar el pedido. El servicio web tardará mucho tiempo, quizás hasta cinco minutos, en procesar completamente el pedido. Por supuesto, es inaceptable para la aplicación web esperar cinco minutos entre el momento en el que el usuario realiza el pedido y el momento en el que se muestran los comentarios. Una buena forma de solucionar este problema es utilizar una cola para desacoplar la aplicación web del servicio web.

La aplicación web recibe el pedido y lo escribe en un mensaje en una cola de Azure Service Bus. Una vez que lo ha hecho, puede informar al usuario de que se está tramitando el pedido. El servicio web toma los mensajes de la cola, uno por uno, y los procesa. Cuando el servicio web termina de procesar un pedido, envía una notificación por correo electrónico al usuario para informarle de que se ha pedido el artículo.

Al desacoplar los sistemas, la aplicación web puede funcionar a una velocidad diferente del servicio web, y ambos pueden escalarse por separado para ajustarse a las necesidades de las aplicaciones.

La cola de Service Bus es un mecanismo sencillo. Varias aplicaciones pueden poner mensajes en la cola, pero un mensaje de la cola solo puede procesarse en una aplicación cada vez.

El servicio incluye algunas características inteligentes para trabajar con los mensajes de la cola, como la detección de duplicados y una subcola de mensajes fallidos a la que se transfieren los mensajes cuando no se pueden procesar correctamente.

4.16 Azure Event Hubs

En un mundo como el actual, en donde el software es algo ubicuo, se generan datos desde innumerables aplicaciones, dispositivos y ubicaciones todo el tiempo. Hoy en día, es cada vez más común para las compañías capturar enormes cantidades de esos datos para analizarlos o transformarlos y moverlos para su uso posterior. Azure Event Hubs puede ayudar con esto.

Event Hubs se ha diseñado para la ingesta masiva de datos. Se pueden lanzar millones de mensajes por segundo al servicio y este gestionará los datos sin esfuerzo. Se pueden retener mensajes hasta un máximo de siete días o retenerlos indefinidamente escribiéndolos en un almacén de datos mediante la función de captura de Event Hubs.

Se puede utilizar Event Hubs para filtrar los datos con consultas en el momento en que llegan y enviarlos a un almacén de datos como Azure Cosmos DB. Se puede incluso reproducir mensajes si es necesario.

4.17 Azure IoT Hub

Al igual que Event Hubs, Azure IoT Hub está diseñado para la ingesta masiva de datos. Va dirigido en especial a la gestión del enorme volumen de mensajes de datos procedentes de los dispositivos del Internet de las cosas, como los termostatos inteligentes o los sensores de los coches. Tiene muchas de las mismas propiedades que Event Hubs, como la capacidad de retener mensajes hasta un máximo de siete días y de reproducirlos.

Lo que hace único a IoT Hub es que, además de recibir mensajes de los dispositivos, puede enviarles mensajes. Tiene la capacidad de administrar toda la infraestructura de IoT. Se puede usar IoT Hub para registrar dispositivos e informar de su estado y para administrarlos protegiéndolos y reiniciándolos y enviándoles datos.

4.18 Azure Event Grid

Azure Event Grid ofrece un tipo diferente de mensajería. Event Grid ofrece un servicio de publicación/suscripción totalmente administrado, que conecta con casi todos los servicios de Azure y puede conectar con suscriptores y publicadores personalizados.

Esto es diferente de trabajar con las colas y los temas de Service Bus, donde solo se necesita sondear la cola o el tema para extraer nuevos mensajes. Event Grid envía automáticamente los mensajes a los suscriptores, lo que lo convierte en un servicio de eventos reactivo en tiempo real.

Da servicio dentro y fuera de los eventos de publicación de Azure (por ejemplo, cuando se agrega un nuevo Blob o cuando se agrega un nuevo usuario a una suscripción de Azure). Azure Event Grid puede detectar estos eventos y hacer que estén disponibles para los controladores de eventos y servicios que se han suscrito.

4.19 Azure Active Directory

Una parte importante de la seguridad de su aplicación es la autenticación de los usuarios antes de utilizarla. La autenticación no es fácil de implementar. Hay que almacenar las identidades y las credenciales de los usuarios en algún sitio, implementar la administración de contraseñas, crear un protocolo de enlace de autenticación seguro, etc.

Azure Active Directory proporciona todo esto “out of the box". Las identidades de los usuarios se guardan en Azure Active Directory y los usuarios se autentican en él, de modo que solo se les redirige a la aplicación una vez que se han autenticado. Azure Active Directory se encarga de administrar las contraseñas, lo que incluye situaciones frecuentes como “He olvidado la contraseña”.

Millones de aplicaciones usan Azure Active Directory cada día, entre ellas Azure Portal, Outlook.com y Office 365. Debido a esto, se puede detectar con más facilidad el comportamiento malicioso y responder a él. Por ejemplo, si un usuario inicia sesión en una aplicación desde una ubicación de Europa y un minuto más tarde inicia sesión desde Australia, Azure Active Directory marca esto como un comportamiento malintencionado y pide al usuario credenciales adicionales por medio de una autenticación multifactor.

4.20 Azure Key Vault

Dentro de la arquitectura de seguridad, se necesita un lugar seguro para almacenar certificados, claves y otros secretos. Azure Key Vault lo proporciona. Con Key Vault, se tiene una ubicación central en la que se almacenan los secretos utilizados por las aplicaciones. Pueden ser las credenciales de una connectionstring utilizada por la aplicación. La aplicación obtendría la connectionstring de Key Vault en vez de obtenerla desde el sistema de configuración. De esta forma, los administradores pueden controlar los secretos y los desarrolladores no tienen que ocuparse de ellos. Key Vault también se utiliza para almacenar los certificados, como capa de sockets seguros (TLS), que se usan para proteger el tráfico intercambiado por las aplicaciones a través de HTTPS.

5 Herramientas

El framework de .net dispone de un conjunto de herramientas que facilitan la creación, implementación y administración de aplicaciones y componentes dirigidos a .NET Framework.

La mayoría de las herramientas de .NET Framework que se describen en este punto se instalan automáticamente con Visual Studio.

A continuación, se listan las herramientas más importantes:

Al.exe (Assembly Linker): Genera un archivo que tiene un manifiesto del ensamblado de módulos o archivos de recursos.

Aximp.exe (Importador de controles ActiveX de Windows Forms): Convierte definiciones de tipos de una biblioteca de tipos COM para un control ActiveX en un control de Windows Forms.

Caspol.exe (Herramienta de la directiva de seguridad de acceso del código): Esta herramienta le permite ver y configurar la directiva de seguridad correspondiente a los niveles de directiva de equipo, de usuario y de empresa.

Cert2spc.exe (Herramienta de prueba de certificados del publicador de software): Crea un certificado de publicador de software (SPC) a partir de uno o varios certificados X.509. Esta herramienta sólo se utiliza para pruebas.

Certmgr.exe (Herramienta de administración de certificados): Administra certificados, listas de certificados de confianza (certificate trust lists, CTL) y listas de revocaciones de certificados (certificate revocation lists, CRL).

Clrver.exe (herramienta de versión de CLR): Notifica todas las versiones instaladas de Common Language Runtime (CLR) en el equipo.

CorFlags.exe (Herramienta de conversión de CorFlags): Le permite configurar la sección CorFlags del encabezado de una imagen ejecutable portátil (PE).

Fuslogvw.exe (Visor de registro de enlaces de ensamblados): Muestra información sobre los enlaces de ensamblado para ayudarle a diagnosticar por qué .NET Framework no puede encontrar un ensamblado en tiempo de ejecución.

Gacutil.exe (Herramienta Caché global de ensamblados): Le permite ver y manipular el contenido de la caché global de ensamblados y de la memoria caché de descarga.

Ilasm.exe (Ensamblador de IL): Genera un archivo ejecutable portable (PE) a partir de lenguaje intermedio (IL). Se puede ejecutar el archivo ejecutable resultante para determinar si IL funciona de la manera esperada.

Ildasm.exe (Desensamblador de IL): Toma un archivo portable ejecutable (PE) que contiene código de lenguaje intermedio (IL) y crea un archivo de texto que se puede usar como entrada para el Ensamblador de IL (Ilasm.exe).

Installutil.exe (Herramienta Installer): Le permite instalar y desinstalar recursos de servidor mediante la ejecución de los componentes del instalador en un ensamblado especificado.

Lc.exe (Compilador de licencias): Lee archivos de texto que contienen información sobre licencias y crea un archivo .licenses que se puede incrustar en un archivo ejecutable de Common Language Runtime como un recurso.

Mage.exe (Herramienta de generación y edición de manifiestos): Le permite crear, editar y firmar manifiestos de aplicación y de implementación. Como herramienta de la línea de comandos, Mage.exe se puede ejecutar desde scripts por lotes y desde otras aplicaciones basadas en Windows, incluidas las aplicaciones ASP.NET.

MageUI.exe (Herramienta de generación y edición de manifiestos, cliente gráfico): Admite la misma funcionalidad que la herramienta de línea de comandos Mage.exe, pero usa una interfaz de usuario (IU) basada en Windows.

MDbg.exe (depurador de línea de comandos .NET Framework): Ayuda a los proveedores de herramientas y desarrolladores de aplicaciones encontrar y corregir errores en los programas que tienen como destino Common Language Runtime de .NET Framework. Esta herramienta utiliza el runtime de la API de depuración para proporcionar servicios de depuración.

Mgmtclassgen.exe (Generador de clases fuertemente tipadas para administración): Le permite generar una clase administrada enlazada en tiempo de compilación para una clase especificada de Instrumental de administración de Windows (WMI).

Mpgo.exe (Herramienta de optimización guiada por perfiles administrados): Permite adaptar ensamblados de imagen nativa mediante escenarios de usuario final comunes. Mpgo.exe posibilita la generación y el consumo de datos de perfil para ensamblados de aplicación de imagen nativa (no los ensamblados de .NET Framework) mediante escenarios de aprendizaje seleccionados por el desarrollador de aplicaciones.

Ngen.exe (Generador de imágenes nativas): Mejora el rendimiento de las aplicaciones administradas mediante el uso de imágenes nativas (archivos que contienen código máquina compilado específico del procesador). El runtime puede usar imágenes nativas de la memoria caché en lugar de usar el compilador Just-In-Time (JIT) para compilar el ensamblado original.

Peverify.exe (Herramienta PEVerify): Le ayuda a comprobar si su código del lenguaje intermedio de Microsoft (MSIL) y los metadatos asociados cumplen los requisitos de seguridad de tipos.

Regasm.exe (Herramienta de registro de ensamblados): Lee los metadatos de un ensamblado y agrega las entradas necesarias al Registro. Esto permite a los clientes COM aparecer como clases de .NET Framework.

Regsvcs.exe (Herramienta de instalación de servicios de .NET): Carga y registra un ensamblado, genera e instala una biblioteca de tipos en una aplicación especificada de la versión de COM+ 1.0 y configura servicios que ha agregado a una clase mediante programación.

Resgen.exe (generador de archivos de recursos): Convierte archivos de texto (.txt o .restext) y archivos de formato de recursos basado en XML (.resx) en archivos binarios de Common Language Runtime (.resources) que se pueden incrustar en un archivo ejecutable binario en tiempo de ejecución o compilar en ensamblados satélite.

SecAnnotate.exe (Herramienta Anotador de seguridad de .NET): Identifica las partes SecurityCritical y SecuritySafeCritical de un ensamblado.

SignTool.exe (Herramienta de firma): Firma digitalmente los archivos, comprueba las firmas de los mismos y les agrega una marca de tiempo.

Sn.exe (Herramienta de nombre seguro): Ayuda a crear ensamblados con nombres seguros. Esta herramienta proporciona opciones para la administración de claves, así como para la generación y comprobación de firmas.

SOS.dll (Extensión de depuración de SOS): Le ayuda a depurar programas administrados en el depurador WinDbg.exe y en Visual Studio al proporcionar información acerca del entorno interno de Common Language Runtime.

SqlMetal.exe (Herramienta de generación de código): Genera código y asignaciones para el componente LINQ to SQL de .NET Framework.

Storeadm.exe (herramienta de almacenamiento aislado): Administra el almacenamiento aislado; proporciona opciones para confeccionar una lista de los almacenes de usuario y eliminarlos.

Tlbexp.exe (Exportador de la biblioteca de tipos): Genera una biblioteca de tipos que describe los tipos definidos en un ensamblado de Common Language Runtime.

TlbImp.exe (Importador de la biblioteca de tipos): Convierte las definiciones de tipos encontradas en una biblioteca de tipos COM en las definiciones equivalentes en un ensamblado de Common Language Runtime.

Winmdexp.exe (Herramienta de exportación de metadatos de Windows Runtime): Exporta un ensamblado de .NET Framework que se compila como un archivo .winmdobj en un componente de Windows Runtime, que se empaqueta como un archivo .winmd que contiene los metadatos y la información de implementación de Windows Runtime.

Winres.exe (Editor de recursos de Windows Forms): Le ayuda a traducir recursos de la interfaz de usuario (IU) (archivos .resx o .resources) usados por Windows Forms. Puede traducir cadenas y, a continuación, ajustar el tamaño, mover y ocultar los controles para dar cabida a las cadenas traducidas.

6 Persistencia

La persistencia se refiere a la característica del estado que sobrevive al proceso que lo creó. En la práctica esto se consigue guardando el estado en algún tipo de sistema de almacenamiento permanente.

En .NET existen muchos mecanismos para implementar la persistencia, cada uno con sus características específicas. En cada caso se debería elegir el mecanismo que mejor se adapte a los requisitos teniendo además en cuenta otros factores como los siguientes:

- Durabilidad. Por ejemplo, Single Storage Service de Amazon AWS tiene una durabilidad del 99.999999999% anual, lo que significa que como media se pierde un objeto de cada cien mil millones al año.

- Disponibilidad. Que se mide como el porcentaje del tiempo en el que es posible recuperar el objeto.
- Rendimiento. Se refiere al tiempo empleado en acceder a un objeto.
- Consistencia. Se refiere a la relación entre lo que se escribe y lo que se lee. En un modelo de consistencia fuerte se garantiza que siempre se lee la última versión de un objeto. Sin embargo, en niveles de consistencia eventual se puede leer una versión anterior del objeto.
- Escalabilidad. Se refiere a la capacidad del sistema a funcionar correctamente con volúmenes de datos y número de accesos crecientes. Es decir, hasta cuánto puede aguantar el sistema cuando aumentamos su carga.
- Robustez. Se refiere a la capacidad de que los datos sobrevivan sin corromperse a pesar de fallos en el sistema como el cuelgue o un fallo catastrófico del sistema operativo o una pérdida de energía del sistema.
- Funcionalidad. Las bases de datos relacionales como SQL Server u Oracle, proporcionan mucha funcionalidad a la hora de recuperar y actualizar la información. Mientras que, en el otro extremo, un simple archivo de texto proporciona una funcionalidad muy limitada.
- Sencillez y facilidad de uso. Siempre debería tenderse a elegir el sistema más sencillo y fácil de utilizar que cumpla con los requisitos. No se debería usar un sistema muy complejo para persistir por ejemplo unos objetos de configuración. Con un simple archivo de texto sería suficiente.

A continuación, se enumeran algunos de los mecanismos de persistencia que se podrían utilizar en aplicaciones .NET:

- Archivos de texto.
    - Estos pueden ser simples archivos csv, en NuGet existen librerías especializadas para leer y escribir en archivos csv como CsvHelper.
    - Archivos XML. En .NET framework existen clases en el espacio de nombres System.Xml y System.Xml.Linq para manipular xml. Además, existe la posibilidad de serializar y deserializar objetos en Xml con XmlSerializer, DataContractSerializer y NetDataContractSerializser.
    - Archivos Json. En NuGet se encuentra la librería Json.Net que permite manipular JSON mediante un modelo de objetos a la vez que proporciona un mecanismo para serializar y deserializar objetos en JSON.
- Archivos Excel. Pueden manipularse con paquetes NuGet como ClosedXml o mediante “Microsoft Access Database Engine”.
- Bases de datos relacionales integradas (embedded). Estas bases de datos se ejecutan en el mismo proceso que la aplicación, no suele ser muy buena idea compartirlas al mismo tiempo entre distintos procesos y menos aún situarlas en carpetas compartidas de la red. Las más destacadas son:

  - Bases de datos de Microsoft Access. Es posible manipular este tipo de bases de datos instalando el "Microsoft Access Database Engine" y accediendo a ellas mediante Ole DB client de ADO.NET. Una ventaja de estas bases de datos es que pueden transportarse y abrirse con Microsoft Access. Como inconveniente sufren de poca robustez y no son multiplataforma.
  - Bases de datos de SQLite. Con el paquete Nuget System.Data.SQLite.Core y ADO.NET puede accederse a esta base de datos integrada. Estas bases de datos son multiplataforma, pueden leerse en cualquier sistema operativo sin cambios, incluso en Android, iOS, Linux y OSX. Son muy robustas y eficiente, pudiendo sobrevivir a pérdidas de energía y fallos catastróficos del sistema operativo.

- Bases de datos de almacenamiento estructurado integrado como ESENT. Con el paquete NuGet ManagedEsent pueden manipularse este tipo de bases de datos desde .NET cuyo motor está presente en todas las versiones de Windows. Se trata de un motor de base de datos muy eficiente y robusto utilizado como almacenamiento en muchas partes de Windows como el directorio activo o Microsoft Exchange.
- Almacenes integrados de clave valor. Existen varios sistemas de almacenamiento clave-valor integrados accesibles desde .NET entre los que se puede destacar:
  - LMDB. Se trata de un motor de almacenamiento clave-valor integrado basado en árboles b+ y memory mapped files muy eficiente y robusto que puede utilizarse desde .NET con el paquete Nuget LightningDB.
  - RocksDb. Se trata de un motor de almacenamiento clave-valor integrado con un diseño en árbol LSM (log-structured-merge) basado en LevelDb de Google y mejorado notablemente por Facebook. Puede usarse en .NET mediante el paquete NuGet RocksDbSharp.
- Bases de datos relacionales con arquitectura cliente-servidor. Quizá se trate del mecanismo de persistencia más empleado dentro de las aplicaciones .NET de línea de negocio. Entre ellos cabe destacar los siguientes:
  - Microsoft SQL Server. Accesible mediante el proveedor SqlClient, SQL Server es un servidor de bases de datos de alto rendimiento, robusto, con muy buenas capacidades de alta disponibilidad y gran funcionalidad.
  - ORACLE. Es el servidor de base de datos más popular, quizá algo más difícil de usar que SQL Server, pero con un rendimiento, funcionalidad y capacidades impresionante. ORACLE proporciona el proveedor ADO.NET que permite conectarnos a las bases de datos de ORACLE.
  - MySQL. Con versiones open source y de pago. MySQL es uno de los motores de base de datos más populares accesible desde .NET mediante el proveedor ADO.NET MySql.Data.MySqlClient descargable como paquete NuGet MySql.Data

  - Postgres. Quizá el mejor motor de bases de datos relacionales de código abierto con una funcionalidad incomparable. Accesible desde .NET mediante el paquete Nuget Npgsql.

- Servidores no SQL. Con una funcionalidad limitada y una consistencia inferior a las bases de datos relacionales, estos servidores de almacenamiento que pueden ser clave-valor, documentales y de otros tipos proporcionan una escalabilidad y rendimiento superiores. Entre ellos podemos destacar:
  - Redis. Usado frecuentemente como sistema de caché distribuido es accesible desde .NET mediante el paquete NuGet StackExchange.Redis
  - Amazon DynamoDB. Amazon DynamoDB accesible desde .NET con el paquete NuGet Amazon.DynamoDb, es un servicio de base de datos NoSQL rápido y flexible para todas las aplicaciones que requieren latencias constantes y de meros milisegundos a cualquier escala. Se trata de una base de datos totalmente administrada en la nube, compatible con modelos de almacenamiento de datos de valor de clave y de documentos. Su modelo de datos flexible, desempeño fiable y escalado automático de la capacidad de desempeño lo hacen ideal para móviles, web, juegos, tecnología publicitaria e IoT.
  - Memcached. Accesible mediante el paquete NuGet EnyimMemcached se trata de un sistema de cache de objetos distribuido de alto rendimiento y de código abierto.
  - Azure Cosmos DB. Azure Cosmos DB es la base de datos multimodelo de distribución global de Microsoft. Azure Cosmos DB permite escalar de forma elástica e individual el rendimiento y el almacenamiento en cualquiera de las regiones geográficas de Azure. Ofrece garantía de rendimiento, latencia, disponibilidad y coherencia con acuerdos de nivel de servicio (SLA) integrales.
  - Mongo DB. Es la base de datos No SQL orientada a documentos de código abierto más popular. Es muy eficiente y escalable, sencilla de utilizar. Recientemente ha superado alguna de las anomalías de consistencia que presentaba en los Jepsen tests y promete en la próxima versión 4.0 transacciones multidocument. Desde .NET puede accederse mediante el paquete NuGet MongoDB.Driver. MongoDB se basa en el motor de almacenamiento WiredTiger escrito en C que proporciona varias estructuras de almacenamiento como árboles B+ y LSM, además de proporcional MVCC (Multiversion Concurrency Control). Desde el punto de vista del teorema CAP, Mongo DB se sitúa en la zona CP.
  - RavenDb. Es Una base de datos documental distribuida con alta disponibilidad de código abierto multiplataforma escrita en C#. Desde el principio ha admitido transacciones multidocumento y permite búsquedas sofisticadas basadas en índices Lucene. La última versión, la versión 4.0 ha logrado aumentar el rendimiento en un factor de 10.0 gracias, en parte, a su nuevo motor de almacenamiento: Voron.

  - CouchDb. Es una base de datos documental distribuida con capacidad de replicación entre centros de datos e incluso contra bases de datos PouchDb en dispositivos móviles.
  - CouchBase. Es una base de datos documental distribuida muy escalable con una gran funcionalidad de consulta comparada con otras bases de datos documentales y capacidad de replicación con dispositivos móviles.
- Otros tipos de almacenamiento en la nube:
  - S3 de Amazon AWS: proporciona almacenamiento de archivos de alta durabilidad.
  - Azure Blob Storage, proporciona almacenamiento masivo escalable de datos no estructurados (archivos).
  - Azure Table Storage, almacenamiento estructurado en la nube, alta disponibilidad y durabilidad.

Como quiera que el modo de persistencia más común entre las aplicaciones de línea de negocio son las bases de datos relacionales, históricamente se ha hecho un gran esfuerzo por facilitar el acceso a las bases de datos desde lenguajes de programación orientados a objetos. Como resultado de ese esfuerzo han nacido múltiples ORM's y micro ORMS de los que cabe destacar:

- Entity Framework. Es un ORM de código abierto desarrollado por Microsoft que proporciona una gran abstracción sobre la base de datos. Esa abstracción hace fácil al programador manipular los datos de la base de datos, pero al mismo tiempo el rendimiento se ve reducido en algunos casos.
- NHibernate. Es la versión .NET del popular ORM Hibernate de Java. Al igual que Entity Framework proporciona un gran nivel de abstracción.
- Dapper. Es un micro ORM desarrollado por StackExchange muy eficiente y simple.
- EntityLite. Es un micro ORM más completo que Dapper con mucha más funcionalidad, pero que sin llegar a proporcionar la abstracción de los ORM completos, y con un enfoque relacional, proporciona un rendimiento sorprendente.

7 Seguridad

7.1 Introducción

.NET Framework dispone de varios mecanismos para proteger recursos y código del acceso por parte de usuarios y código no autorizados:

- ✓ Seguridad ASP.NET de aplicaciones Web: ofrece una manera de ayudar a limitar el acceso a un sitio comparando las credenciales autenticadas (o representaciones de las mismas) con los permisos del sistema de archivos de

Microsoft Windows NT o con un archivo XML que contiene la lista de usuarios autorizados, funciones autorizadas o verbos HTTP autorizados.

- ✓ Seguridad de acceso a código utiliza permisos para ayudar a limitar el acceso que el código tiene a recursos y operaciones protegidos. Ayuda a proteger los sistemas de equipos de código móvil malicioso y ayuda a permitir la ejecución segura del código móvil. La seguridad de acceso a código junto con las directivas que la rigen se denominan seguridad basada en evidencias.

7.2 Seguridad ASP.NET de aplicaciones web

En la mayoría de los sitios Web es necesario restringir el acceso de forma selectiva a determinadas partes del sitio. ASP.NET, conjuntamente con Servicios de Microsoft Internet Information Server (IIS), puede autenticar las credenciales del usuario como nombres y contraseñas mediante los métodos de autenticación siguientes:

- ✓ Windows: básica, implícita, y Autenticación de Windows integrada (NTLM o Kerberos).
- ✓ Autenticación mediante Microsoft Passport
- ✓ Autenticación mediante formularios
- ✓ Autenticación mediante certificados de cliente

Además, Internet Information Services (IIS) puede conceder o negar el acceso en función de la dirección IP o del nombre de host del usuario. Cualquier autorización de acceso posterior se realiza mediante la autorización de la dirección URL del permiso de acceso al sistema de archivos NTFS.

ASP.NET controla el acceso a la información de los sitios comparando las credenciales autenticadas, o representaciones de las mismas, con los permisos del sistema de archivos de Microsoft Windows NT o con un archivo XML que contiene la lista de usuarios autorizados, funciones autorizadas (grupos) o verbos HTTP autorizados.

Para ayudar a proteger la seguridad de una aplicación ASP.NET, se deben realizar las dos funciones principales que se describen en la siguiente tabla.

Función de seguridad	Descripción
Autenticación	Ayuda a comprobar que el usuario es precisamente quien dice ser. La aplicación obtiene las credenciales (diversas formas de identificación, como nombre y contraseña) de un usuario, y las valida consultando a una autoridad determinada. Si las credenciales son válidas, se considera a la entidad que ha enviado las credenciales como una identidad autenticada.
Autorización	Limita los derechos de acceso mediante la concesión o negación de permisos específicos a una identidad

	autenticada.

Si se solicita una dirección URL que contiene una aplicación ASP.NET, la información sobre la autenticación y la solicitud se entrega a la aplicación. ASP.NET proporciona dos tipos adicionales de autenticación, que se describen en la tabla siguiente.

Proveedor de autenticación de ASP.NET	Descripción
Autenticación mediante formularios	Sistema que redirige las solicitudes no autenticadas a un formulario HTML mediante la redirección del cliente HTTP. El usuario proporciona las credenciales y envía el formulario. Si la aplicación autentica la solicitud, el sistema emite un vale de autenticación en una cookie que contiene las credenciales o una clave para readquirir la identidad. Las solicitudes posteriores se emiten con la cookie en los encabezados de la solicitud; se autentican y se autorizan en un controlador ASP.NET mediante el método de validación que especifique el programador de la aplicación.
Autenticación mediante Passport	Servicio de autenticación centralizado proporcionado por Microsoft que ofrece a los sitios Web suscritos servicios de perfil básico y un inicio de sesión único.

La configuración de ASP.NET, de la que forma parte la seguridad, tiene una arquitectura jerárquica. Toda la información de configuración de ASP.NET se encuentra en los archivos denominados Web.config y Machine.config. Web.config puede colocarse en los mismos directorios que los archivos de la aplicación. El archivo Machine.config se encuentra en el directorio Config de la raíz de la instalación. Los subdirectorios heredan las configuraciones del directorio, a no ser que se reemplacen por un archivo Web.config del subdirectorio. En un archivo Web.config, hay secciones para cada categoría principal de las funcionalidades de ASP.NET.

8 Desarrollo de aplicaciones web en .NET

8.1 Arquitectura detallada de una aplicación web distribuida

Basándonos en la ilustración presentada al final de este punto vemos los distintos elementos que intervienen:

- ✓ **Diseño de capas de presentación**
- ✓ Componentes de Interfaz de Usuario (IU)
  - Interaccionan con el usuario
  - Cuando un usuario interactúa con un elemento de la interfaz, genera un evento que llama al código de una función de control
  - Restringen tipos de entrada de usuario, validan datos…
  - No inicializan transacciones
  - Permiten procesado, formateo y validación de los datos
  - Pueden personalizar el aspecto de la aplicación
  - Implementados con ASP.NET
- ✓ Componentes de Proceso de Usuario
  - Se implementan cuando la interactuación del usuario con la aplicación puede seguir un proceso predecible
  - Se suelen implementar como clases .NET que exponen métodos a los que pueden llamar las interfaces de usuario
  - Abstraen la funcionalidad de procesamiento y adquisición de datos
  - Organizan y sincronizan las acciones del usuario
  - Mantienen el flujo del proceso y el estado
  - Pueden proporcionar características de tipo “guardar y continuar más adelante”
  - Distintas IU (Interfaces de Usuario) pueden compartir un mismo proceso de usuario (web forms y windows forms)
- ✓ **Diseño de capas empresariales**
  - La parte más importante de la aplicación es la funcionalidad que proporciona
  - Para procesos empresariales más complejos se requieren varios pasos y transacciones de ejecución larga -> hay que organizar las tareas y almacenar el estado
  - Se puede diseñar la lógica para su uso directo por la capa de presentación; o bien, encapsular como servicio y llamar a través de una interfaz de servicios
- ✓ Flujos de Trabajo Empresariales (Workflows)
  - Manejan una serie de pasos complejos en el servidor
  - Usan eventos, ramas de decisión, estados, etc. para decidir el camino
  - Realizan llamadas a los componentes empresariales correspondientes en cada paso

- Se utilizan para largas y complejas operaciones
- Pueden exponer interfaces para establecer conversaciones o contratos con otros servicios
- Implementados mediante BizTalk Orchestation

✓ Componentes Empresariales

- Implementan las reglas de negocio
- Realizan las tareas requeridas por las reglas de negocio
- Desarrollan como .NET Enterprise Services (componentes COM+) o Web Services
- Inician o participan en las transacciones
- Aceptan y devuelven estructuras de datos simples o complejas
- Deben exponer funcionalidad independiente de los almacenes de datos y los servicios necesarios para realizar la tarea
- Son invocados por la capa de proceso de usuario, interfaces de servicio y otros procesos empresariales (normalmente con datos empresariales)
- Pueden llamar a componentes lógicos de acceso a datos para recuperar y actualizar datos de la aplicación
- Pueden llamar a servicios externos a través de agentes de servicios
- Pueden llamar a otros componentes empresariales

✓ Entidades Empresariales

- Representan los datos que se pasan entre los componentes
- Los formatos de los datos pueden ser muy variados (XML, Data Reader, conjunto de datos…)
- Suelen tener estructura compleja (Ej: entidad factura)
- El formato depende del modo en el que se trabaje con ellos
- Se modelan mediante objetos o documentos (XML), dependiendo de las necesidades de la aplicación

✓ Interfaz de servicios

- Se usan para ofrecer la funcionalidad de los componentes empresariales a otros componentes
- Proporcionan un punto de entrada para que los clientes llamen y se abstraigan de la implementación interna
- Son únicamente elementos tipo 'fachada', que controlan los servicios de asignación y transformación para permitir la comunicación con un servicio

- Exponen un conjunto de métodos a los que se puede llamar de forma individual, o en secuencia específica
- Puede haber varias para una funcionalidad concreta
- Los requisitos de formato de comunicación, esquema de seguridad e intercambio de datos se determinan como parte del contrato publicado por el servicio

✓ Agentes de Servicios

- Permiten a los componentes empresariales acceder un servicio externo
- Para ello, encapsulan el acceso a un servicio
- Aíslan la implementación de los procesos empresariales de la implementación de formato de datos
- Se encargan de la comunicación y la conversión de datos
- Además, pueden hacer validación básica de datos intercambiados con el servicio
- Pueden autorizar el acceso al servicio
- Pueden definir la seguridad adecuada al servicio
- Pueden proporcionar información sobre el nivel de servicio

✓ **Diseño de capas de datos**

✓ Componentes lógicos de acceso a datos

- Deben usar ADO.NET o bien XML
- Pueden atacar a uno o más almacenes de datos. Resulta más sencillo obtener el acceso de un único origen por motivos de mantenimiento
- Encapsulan todas las operaciones de almacenamiento: conexiones, consultas, escrituras, etc.
- Llevan a cabo asignaciones y transformaciones simples de argumentos de entrada y salida (abstraen la lógica empresarial de los esquemas de base de datos)
- Pueden realizar operaciones sobre tablas seleccionadas
- No deben realizar transacciones, sino participar
- No deben invocar a otros componentes lógicos de acceso a datos
- Funcionalidad típica:
- Funciones CRUD
- Consultas de un gran número de tablas con propósito de sólo lectura
- Acciones que actualizarán y devolverán datos
- Devolución de metadatos relacionados con los esquemas de entidades

- Paginación de las interfaces de usuario que requisen subconjuntos de datos

